



操作系统原理

Operating Systems Principles

陈鹏飞
计算机学院



中山大學

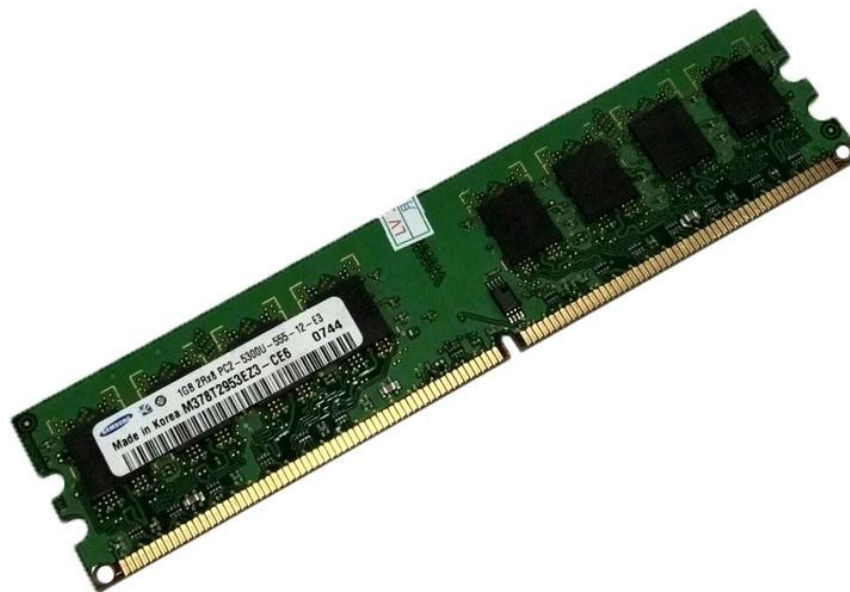
SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



第九讲 — 内存管理策略





目标

- 讨论内存管理的多种方法；
- 内存分页和分段的概念；
- 内存硬件的各种组织方法；
- 进程内存的分配的各种技术；
- 详细讨论现代计算机系统的分页机制；
- 动态链接与共享；



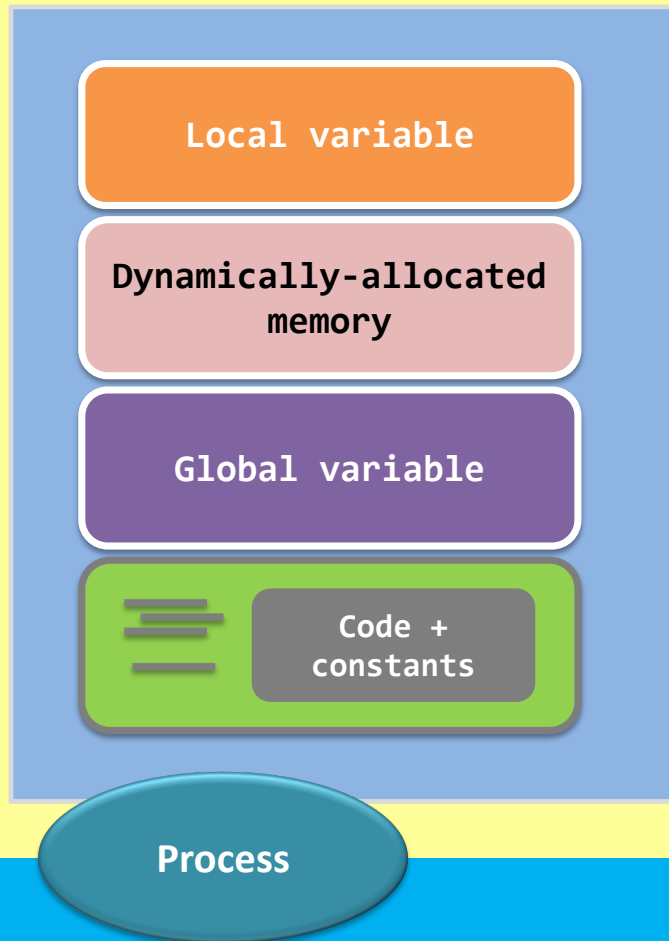
Why we need memory management

- The running program code requires memory
 - Because the CPU needs to fetch the instructions from the memory for execution
- We must keep several processes in memory
 - Improve both CPU utilization and responsiveness
 - Multiprogramming

It is required to efficiently manage the memory



Part 1: User-space memory



Do you remember this?

- Content of a process (in user-space memory)

How does each part use the memory?

- From a programmer's perspective

Let's forget about the kernel for a moment. We are going to explore the **user-space memory** first.



Address space

How does a programmer look at the memory space?

- An array of bytes?
- Memory of a process is divided into segments
- This way of arranging memory is called **segmentation**

Stack - Local variables

Heap - Dynamically allocated memory

Data Segment & BSS - Global and static variables

Code + Constant



Address space

```
int main(void) {  
    int *malloc_ptr = malloc(4);  
    char *constant_ptr = "hello";  
  
    printf("Local variable = %15p\n", &malloc_ptr);  
    printf("malloc() space = %15p\n", malloc_ptr);  
    printf("Global variable = %15p\n", &global_int);  
    printf("Code & constant = %15p\n", constant_ptr);  
  
    return 0;  
}
```

\$./addr

```
Local variable = 0xbfa8938c  
malloc() space = 0x915c008  
Global variable = 0x804a020  
Code & constant = 0x8048550  
$ _
```

Note

The addresses are not necessarily the same in different processes

What is the process address space?

Increasing
address

Stack - Local variables

Heap - Dynamically
allocated memory

Data Segment & BSS -
Global and static
variables

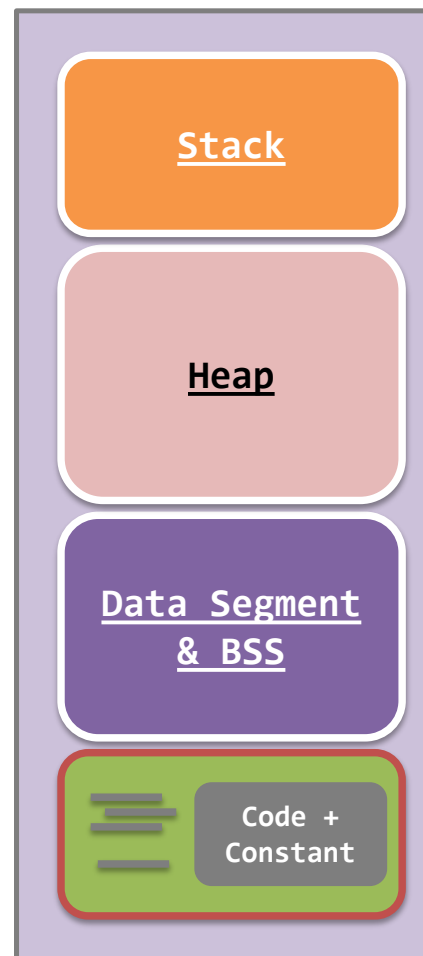
Code + Constant



Program code & constants

```
1 int main(void) {  
2     char *string = "hello";  
3     printf("\nhello\n"      = %p\n", "hello");  
4     printf("String pointer = %p\n", string);  
5     string[4] = '\0';  
6     printf("Go to %s\n", string);  
7     return 0;  
8 }
```

- Constants are stored in code segment.
 - The memory for constants is decided by the program code
 - Accessing of constants are done using addresses (or pointers).
- Codes and constants are both **read-only**.





Data Segment & BSS – properties

```
int global_int = 10;
int main(void) {
    int local_int = 10;
    static int static_int = 10;
    printf("local_int  addr = %p\n", &local_int );
    printf("static_int addr = %p\n", &static_int );
    printf("global_int addr = %p\n", &global_int );
    return 0;
}
```

```
$ ./global_vs_static
local_int  addr = 0xbf8bb8ac
static_int addr = 0x804a018
global_int addr = 0x804a014
$_
```

They are stored next to each other.

This implies that they are **in the same segment!**

Note: A static variable is treated as the same as a global variable!

Stack

Heap

Data Segment
& BSS

Code +
Constant



Data Segment & BSS – locations

```
1 int global_bss;  
2 int global_data = 10;  
3 int main(void) {  
4     static int static_bss;  
5     static int static_data = 10;  
6     printf("global bss = %p\n", &global_bss );  
7     printf("static bss = %p\n", &static_bss );  
8     printf("global data = %p\n", &global_data );  
9     printf("static data = %p\n", &static_data );  
10 }
```

```
$ ./data_vs_bss
```

```
global bss = 0x804a028
```

```
static bss = 0x804a024
```

```
global data = 0x804a014
```

```
static data = 0x804a018
```

```
$ _
```

BSS

Data

Stack

Heap

Data Segment
& BSS

Code +
Constant



Data Segment & BSS – sizes

```
char a[1000000] = {10};

int main(void) {
    return 0;
}
```

Program: data_large.c

```
char a[100] = {10};

int main(void) {
    return 0;
}
```

Program: data_small.c

No optimization.

```
$ gcc -O0 -o data_large data_large.c
$ gcc -O0 -o data_small data_small.c

$ ls -l data_small data_large
```

Guess! Which one is large?

What is the difference between data and BSS?



Data Segment & BSS – sizes

```
char a[1000000] = {10};

int main(void) {
    return 0;
}
```

Program: data_large.c

```
char a[100] = {10};

int main(void) {
    return 0;
}
```

Program: data_small.c

```
$ gcc -O0 -o data_large data_large.c
$ gcc -O0 -o data_small data_small.c

$ ls -l data_small data_large
-rwxr-xr-x ... 1004816 ... data_large
-rwxr-xr-x ... 4916 ... data_small
$_
```

Wow!

The data segment has the required space already allocated.



Data Segment & BSS – sizes

```
char a[1000000];
```

```
int main(void) {  
    return 0;  
}
```

Program: bss_large.c

```
char a[100];
```

```
int main(void) {  
    return 0;  
}
```

Program: bss_small.c

```
$ gcc -O0 -o bss_large bss_large.c
```

```
$ gcc -O0 -o bss_small bss_small.c
```

```
$ ls -l bss_small bss_large
```

```
-rwxr-xr-x ... 4775... bss_large
```

```
-rwxr-xr-x ... 4775... bss_small
```

```
$ _
```

Same size!

To the program, BSS is just a bunch of symbols.
The space is not yet allocated.

The space will be allocated to the process once it starts executing.

This is why BSS is called “*Block Started by Symbol*”.



Data Segment & BSS – limits

How large is the data segment?

```
$ ulimit -a
core file size      (blocks, -c) 0
data seg size       (kbytes, -d) unlimited
.....
$ _
```

In Linux, “**ulimit**” is a built-in command in “**/bin/bash**”.

It sets or gets the system limitations in the current shell.

Does the “unlimited” mean that you can define a global array with large enough size?



Data Segment & BSS – limits

```
#define ONE_MEG (1024 * 1024)

char a[1024 * ONE_MEG];

int main(void) {
    memset(a, 0, sizeof(a));
    printf("1GB OK\n");
}
```

```
#define ONE_MEG (1024 * 1024)

char a[2048 * ONE_MEG];

int main(void) {
    memset(a, 0, sizeof(a));
    printf("2GB OK\n");
}
```

```
$ gcc -Wall -O0 global_2gb.c -o global_2gb
global_2gb.c:6: warning: integer overflow in expression
global_2gb.c:6: error: size of array 'a' is negative
$ _
```

The size of an array is a 32-bit **signed integer**, no matter 32-bit or 64-bit systems. Therefore...



Data Segment & BSS – limits

```
#define ONE_MEG (1024 * 1024)

char a[1024 * ONE_MEG];
char b[1024 * ONE_MEG];
char c[1024 * ONE_MEG];
char d[1024 * ONE_MEG];

int main(void) {
    memset(a, 0, sizeof(a));
    printf("1GB OK\n");
    memset(b, 0, sizeof(b));
    printf("2GB OK\n");
    memset(c, 0, sizeof(c));
    printf("3GB OK\n");
    memset(d, 0, sizeof(d));
    printf("4GB OK\n");
}
```

Program: global_4gb.c

Segmentation fault
why?

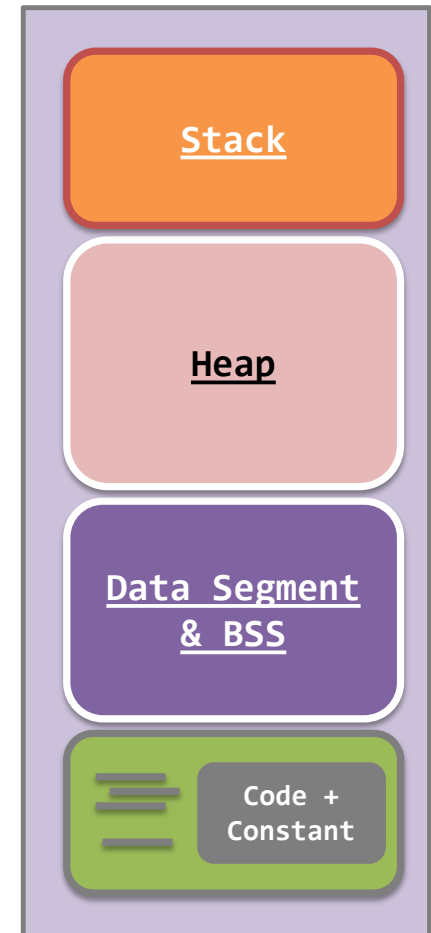
On a **32-bit** Linux system, the user-space **addressing space is around 3GB**.

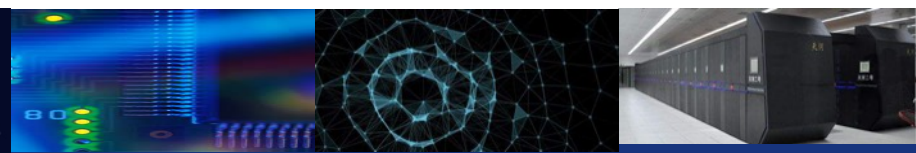
The kernel reserves 1GB addressing space.



Stack – properties

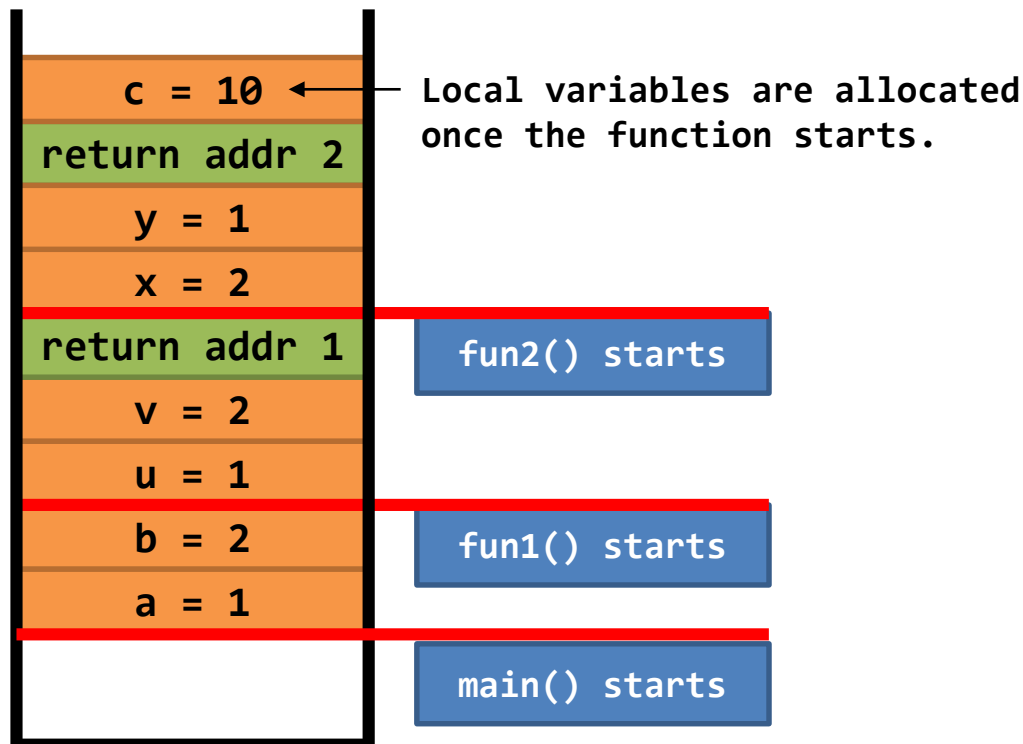
- Stack: FILO
- When a function is called, the local variables are allocated in the stack.
- When a function returns, the local variables are deallocated from the stack.





Stack – push & pop mechanisms

Calling function “**fun2()**” takes place. The CPU has switched to **fun2()**.



```
int fun2(int x, int y) {  
    int c = 10;  
    return (x + y + c);  
}
```

```
int fun1(int u, int v) {  
    return fun2(v, u);  
}
```

```
int main(void) {  
    int a = 1, b = 2;  
    b = fun1(a, b);  
    return 0;  
}
```



Stack – limits

```
$ ulimit -a
core file size      (blocks, -c) 0
data seg size       (kbytes, -d) unlimited
.....
stack size          (kbytes, -s) 8192
.....
$ _
```

So, the limit is:
 $8192 \times 1024 = 8\text{MB}$.

Can you define a local array larger than the limit?

Segmentation
fault

```
$ ulimit -a
core file size      (blocks, -c) 0
data seg size       (kbytes, -d) unlimited
.....
stack size          (kbytes, -s) 8192
.....
$ ulimit -s 81920
```

Now, the limit is:
 $81920 \times 1024 = 80\text{MB}$.

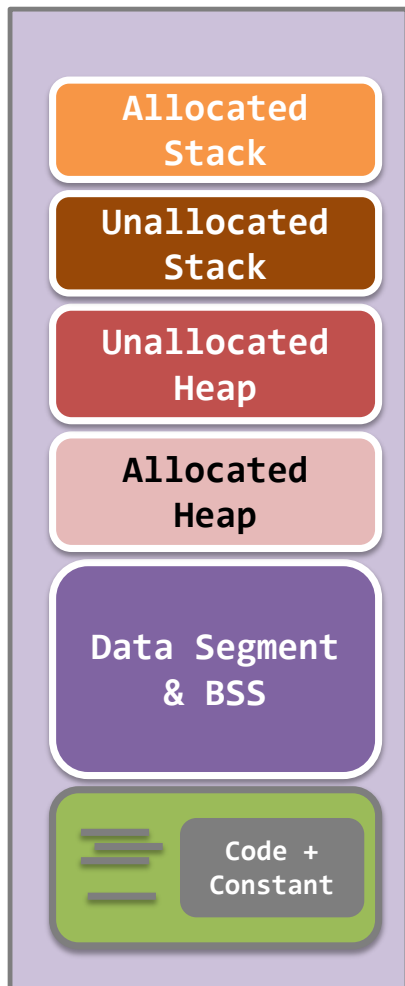


What is segmentation fault?

- Someone must have told you:
 - When you are accessing a piece of memory that is not allowed to be accessed, then the OS returns you an error called – segmentation fault.
- As a matter of fact, how many ways are there to generate a segmentation fault?



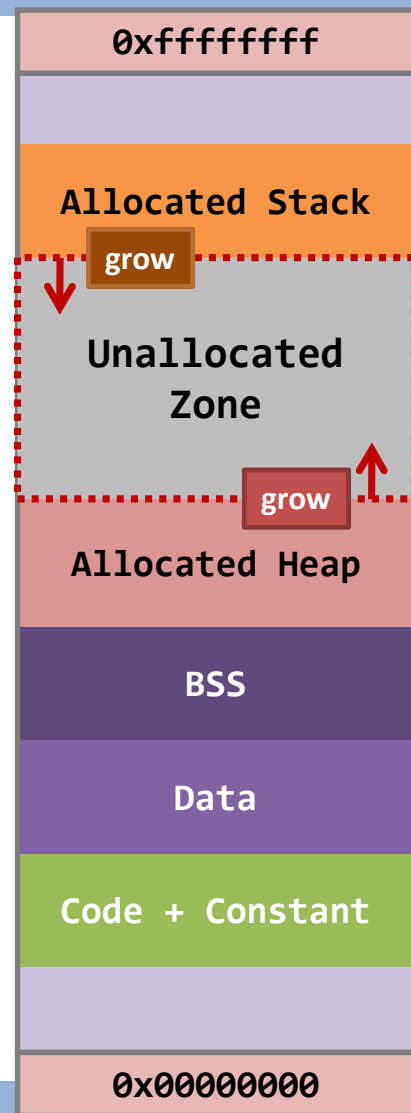
What is segmentation fault?



From illustration to reality...

Forget about the illustration, the memory in a process is separated into **segments**.

So, when you visit a segment in an illegal way, then...**segmentation fault**.





How to “segmentation fault”?

Read	0xffffffff	Write
YES	Unusable	YES
NO	Allocated Stack	NO
YES	Unallocated Zone	YES
NO	Allocated Heap	NO
NO	BSS	NO
NO	Data	NO
NO	Code + Constant	YES
YES	Unusable	YES
	0x00000000	



How to “segmentation fault”?

Read	0xffffffff	Write
YES	Unusable	YES
NO	Allocated Stack	NO
YES	Unallocated Zone	YES
NO	Allocated Heap	NO
NO	BSS	NO
NO	Data	NO
NO	Code + Constant	YES
YES	Unusable	YES
	0x00000000	

Now, we can understand:

```
char *ptr = NULL;  
char c = *ptr;
```

will generates

Segmentation fault

NULL = 0x00000000



*ptr is reading



Summary of segmentation fault

- When you have a **so-called address** (maybe it is just a random sequence of 4 bytes), one of the following cases happens:

See if you have luck...

	Read-only segments	Allocated segments	Unused or unallocated segments
Reading	No problem	No problem	Segmentation fault
Writing	Segmentation fault	No problem	Segmentation fault



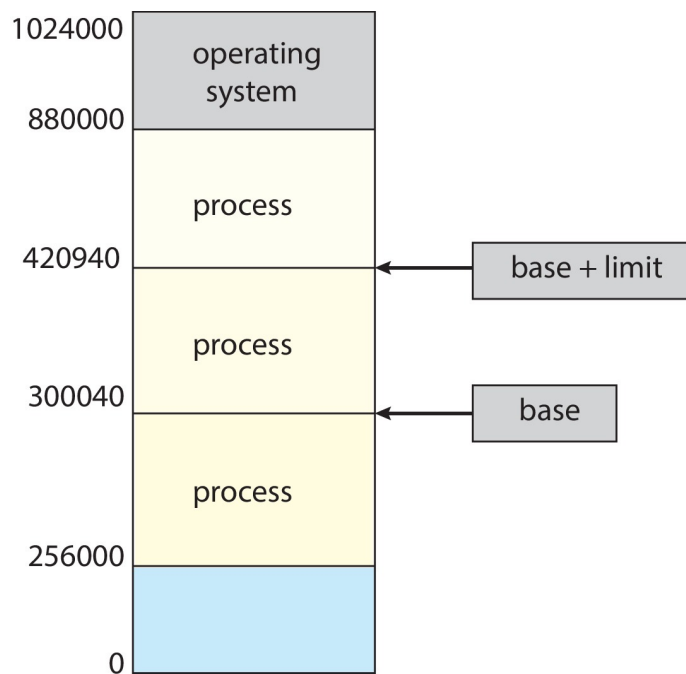
内存管理

- ❖ 程序必须（从磁盘）放入内存并放入进程中才能运行
- ❖ 主存储器和寄存器是CPU可以直接访问的唯一存储器
- ❖ 内存单元只能看到以下数据流：
 - 地址+读取请求，或
 - 地址+数据和写入请求
- ❖ 寄存器访问在一个CPU时钟（或更少）内完成；
- ❖ 主内存访问可能会占用许多周期，从而导致暂停(stall)；
- ❖ 缓存(cache)位于主内存和CPU寄存器之间；
- ❖ 不仅关系内存的访问速度，还要确保操作正确，保护操作系统，通过硬件来实现, CPU不干预CPU对内存的访问；



内存保护

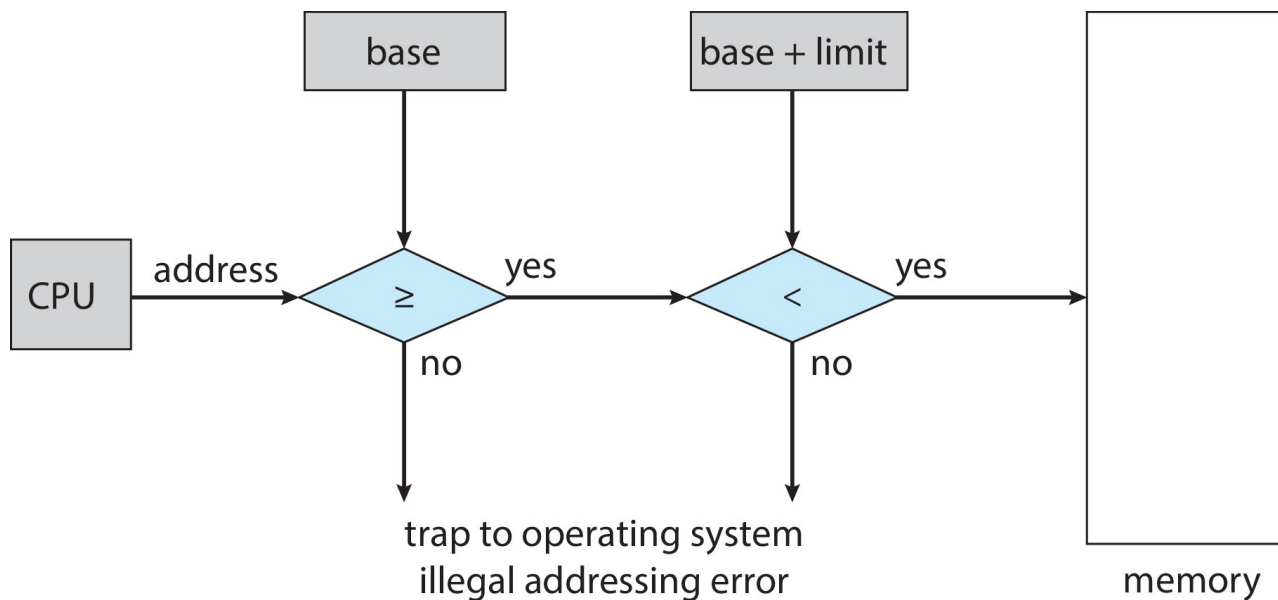
- ❖ 需要限制进程只能访问其地址空间中的那些地址。
- ❖ 可以通过使用一对基址寄存器和界限地址寄存器定义进程的可访问地址；
- ❖ 址空间来提供这种保护，通过CPU硬件实现；





硬件地址保护

- ❖ CPU必须检查在用户模式下生成的每个内存访问，以确保该用户的内存访问量在基地址和界限之间



- ❖ 加载基地址寄存器和界限寄存器的指令具有特权性，只能在内核模式下运行，也只有操作系统可以加载基地址寄存器和界限地址寄存器



地址绑定

- ❖ 磁盘上的程序，准备进入内存执行，形成一个输入队列
 - 如果没有内存管理没有支持，程序必须加载到地址0000
- ❖ 第一个用户进程物理地址始终为0000不方便
 - 如何实现灵活的地址装载？
- ❖ 在程序生命周期的不同阶段以不同方式表示的地址
 - 源代码地址通常是符号地址；
 - 编译代码地址绑定到可重定位地址；
 - 例如，“此模块开头的14字节开始”
 - 链接器或加载程序将可重定位地址绑定到绝对地址（物理地址）
 - 例如，74014
 - 每一次绑定都将一个地址空间映射到另一个地址空间

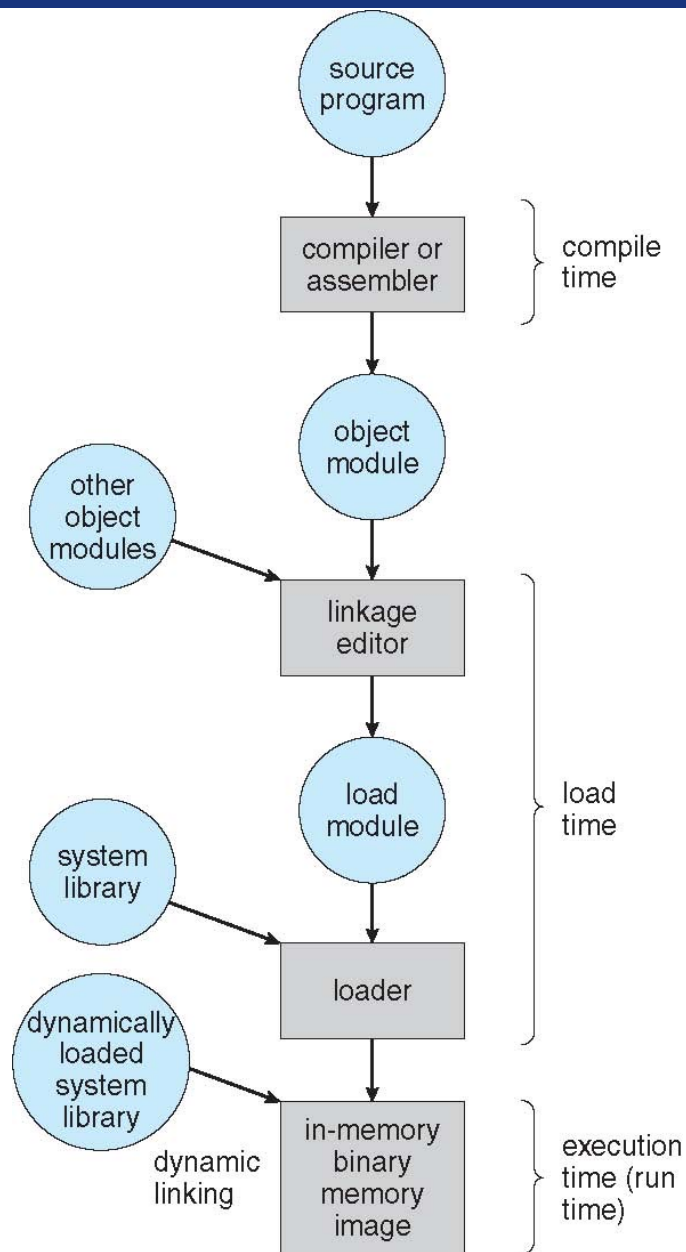


指令和数据绑定到内存

- ❖ 指令和数据到内存的地址绑定可以发生在三个不同的阶段
 - 编译时：如果内存位置先验已知，则可以生成绝对代码；如果起始位置更改，则必须重新编译代码；
 - 加载时：如果编译时内存位置未知，则必须生成可重定位代码；
 - 执行时间：如果进程在执行期间可以从一个内存段移动到另一个内存段，则绑定延迟到运行时；
 - 需要硬件支持地址映射（例如，基寄存器和限制寄存器）



用户程序的处理过程





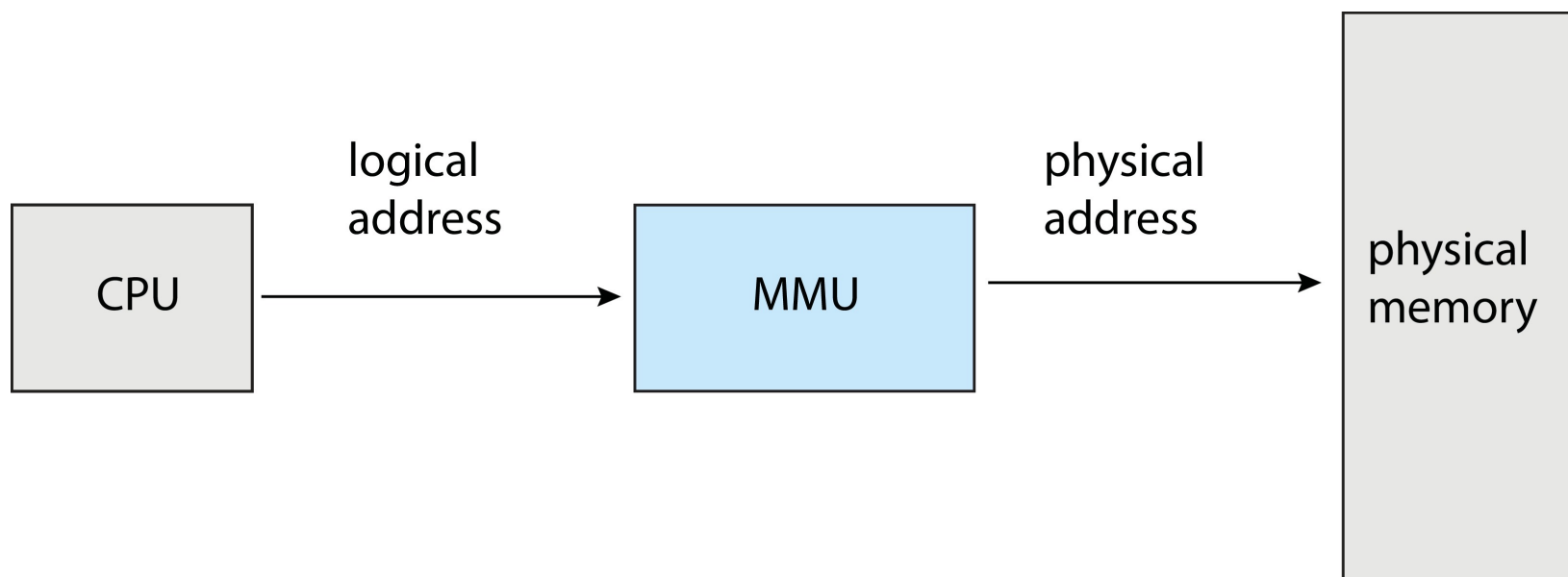
逻辑地址空间和物理地址空间

- ❖ 绑定到单独物理地址空间的逻辑地址空间的概念是正确内存管理的核心;
 - 逻辑地址 - 由CPU生成; 也称为虚拟地址;
 - 物理地址 - 内存单元看到的地址;
- ❖ 编译时和加载时地址绑定方案中的生成相同的逻辑地址和物理地址; 逻辑 (虚拟) 地址和物理地址在执行时地址绑定方案上有所不同;
- ❖ 逻辑地址空间是程序生成的所有逻辑地址的集合;
- ❖ 物理地址空间是程序生成的所有物理地址的集合;



内存管理单元 (MMU)

- ❖ 在运行时, 将虚拟地址映射到物理地址的硬件设备



- ❖ 本章其余部分将介绍许多可能的方法



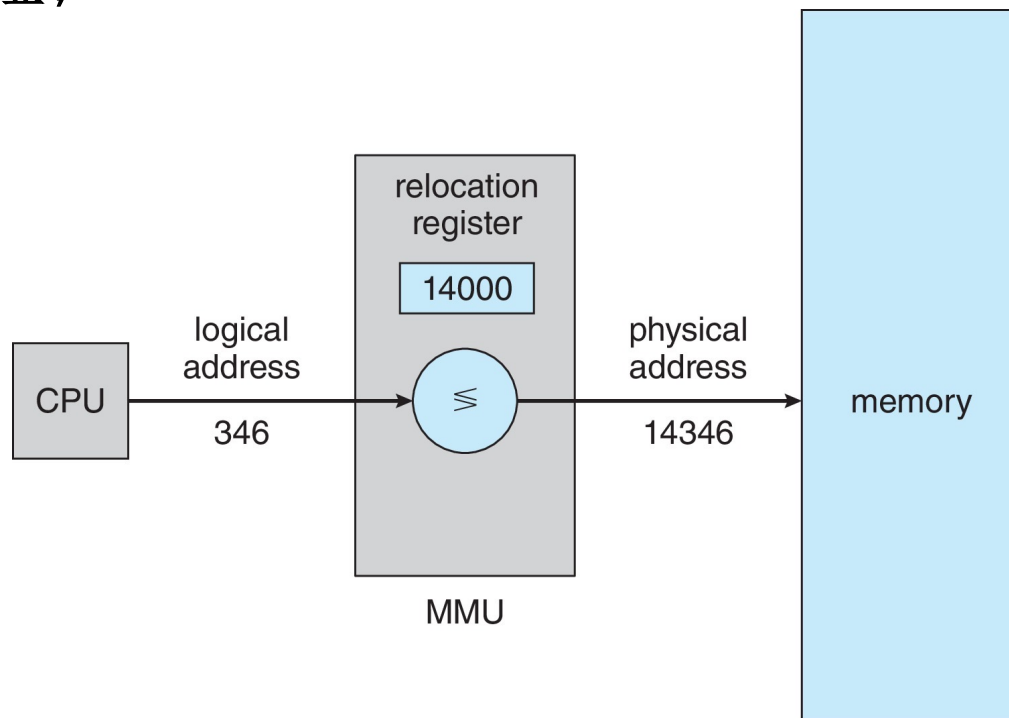
内存管理单元 (MMU)

- ❖ 考虑简单的方案。这是基址寄存器方案的推广；
 - 基址寄存器现在称为重定位寄存器
- ❖ 重定位寄存器中的值在发送到内存时加上用户进程生成的每个地址；
- ❖ 例如：如果基地址为14000，那么对地址346的访问映射位置为：14346。
- ❖ 用户程序处理逻辑地址；它不会看到真正的物理地址；
 - 运行时绑定在内存中的位置被引用时发生；
 - 绑定到物理地址的逻辑地址；



内存管理单元 (MMU)

- ❖ 考虑简单的方案。这是基址寄存器方案的推广。
- 基址寄存器现在称为重定位寄存器
- ❖ 重定位寄存器中的值在发送到内存时添加到用户进程生成的每个地址；





动态加载

- 整个程序需要在内存中才能执行。内存利用率低;
- 动态装载, 在调用程序之前, 不会加载该程序;
- 更好的内存空间利用率, 不加载未使用的程序;
- 程序以可重定位加载格式保存在磁盘上;
- 动态加载的优点: 只有一个程序被需要时才会被加载;
- 当需要大量代码来处理不经常发生的情况时如处理异常情况, 此功能非常有用, 虽然程序很大, 但是可以所加载的部分很小;
- 不需要操作系统的特殊支持
 - 通过程序设计实现;
 - 操作系统可以通过提供库来实现动态加载来提供帮助



动态链接与共享库

- ❖ 静态链接 - 加载程序将系统库和程序代码组合到二进制程序映像中
- ❖ 动态链接 - 链接延迟到运行时;
- ❖ 一小段代码, 存根, 用于定位适当的内存驻留库程序;
- ❖ 存根将自身替换为程序的地址, 并执行程序
- ❖ 操作系统检查程序是否在进程的内存地址中
 - 如果不在地址空间中, 添加到地址空间;
- ❖ 动态链接对于库特别有用, 特别是库的更新;
- ❖ 系统也称为共享库;
- ❖ 考虑维护系统库的适用性
 - 可能需要进行版本控制, 包含在程序和库中;



交换

进程必须在内存中以便执行，不过，进程可以暂时从内存中交换到备份存储或磁盘；交换可能让所有进程的总的物理地址空间超过真实的物理地址空间；

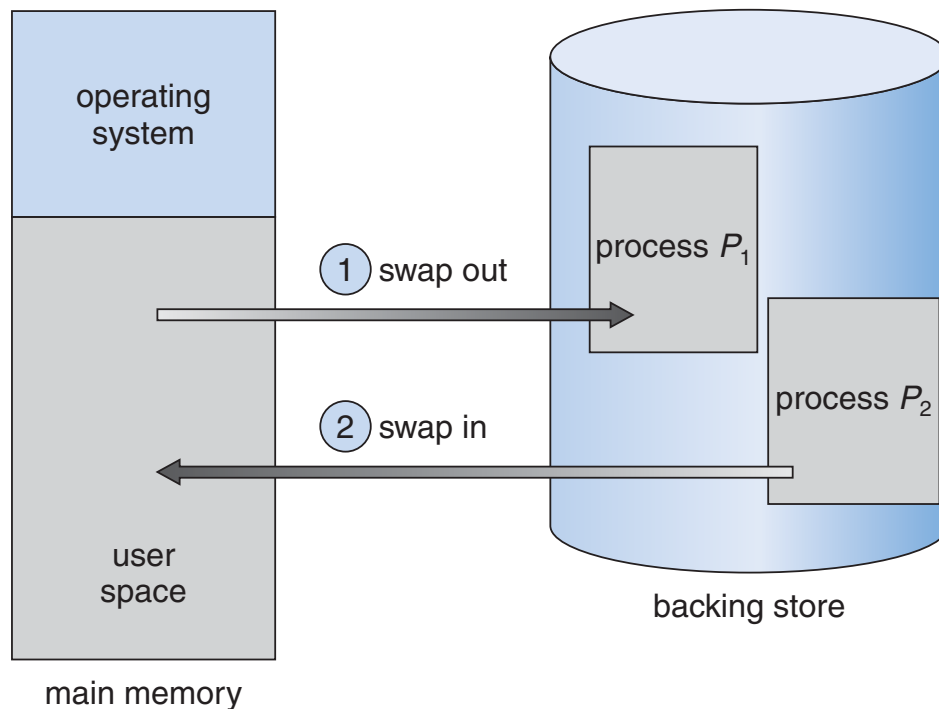


Figure 7.5 Swapping of two processes using a disk as a backing store.



交换

- 标准交换—从内存到备份存储，一般是磁盘；
交换的代价很高；
依赖于内存和磁盘的传输速率；
- 在正常情况下，禁止交换；当空闲时内存低于某个阈值时，启用交换；这些交换通常与虚拟内存一起工作；
- 移动系统的交换：通常禁止交换，如果没有足够的内存，则会终止进程；终止进程之前，会将状态保存到闪存，以便快速恢复；
- 移动系统开发人员应小心分配和释放内存；



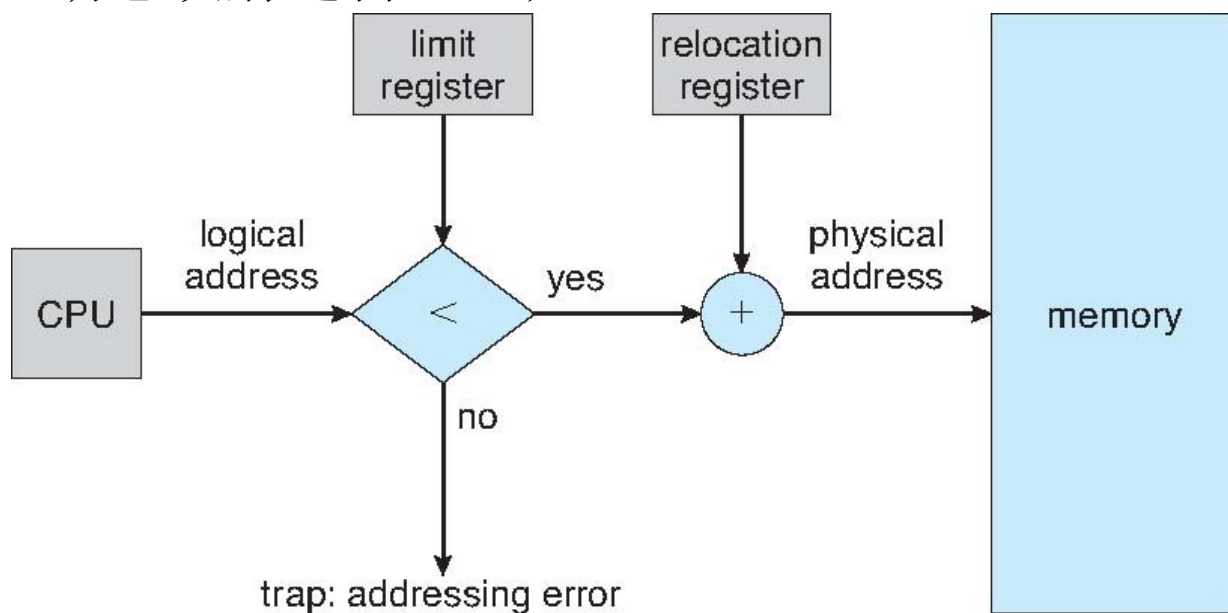
连续分配

- ❖ 主内存必须同时支持操作系统和用户进程
- ❖ 有限的资源，必须有效配置
- ❖ 连续分配是一种早期的方法
- ❖ 主内存通常分为两个分区：
 - 常驻操作系统，通常中断向量保存在低内存中
 - 然后将用户进程保存在高内存中；
 - 每个进程包含在内存的单个连续部分中；



连续分配

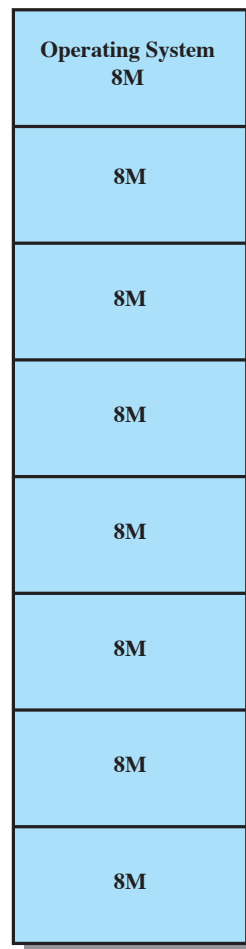
- ❖ 重新定位寄存器，用于保护用户进程彼此不受影响，以及防止更改操作系统代码和数据
 - 基址寄存器包含最小物理地址的值；
 - 界限寄存器包含逻辑地址范围 - 每个逻辑地址必须小于界限寄存器；
 - MMU动态映射逻辑地址；



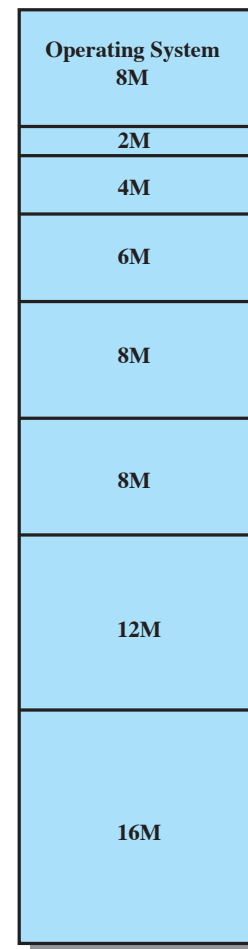


固定分区

- 操作系统占据内存中的固定部分;
- 用户内存空间的最简单方案是对它分区, 以形成若干边界固定的区域;
- 分区大小: 使用大小相等的分区;
使用大小不等的分区;



(a) Equal-size partitions



(b) Unequal-size partitions

Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory



固定分区的缺点

- ❖ 分区数量在系统生成阶段已经确定，因而限制了系统中活动（未挂起）进程的数量；
- ❖ 由于分区是在系统生成阶段事先设置的，因而小作业不能有效利用分区空间

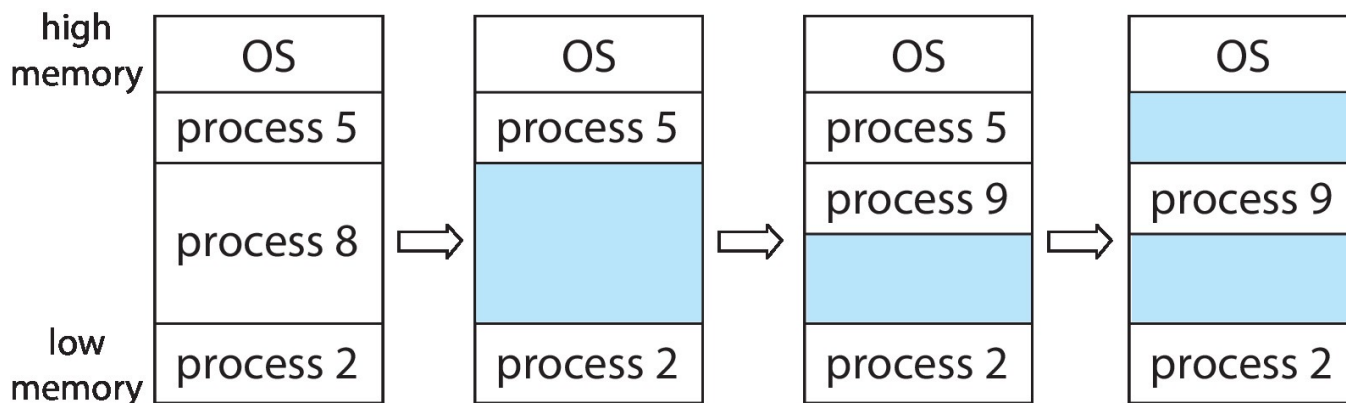




可变分区

❖ 多分区分配

- 多道程序设计程度受分区数量限制;
- 提高效率的可变分区大小 (根据给定进程的需要调整大小)
- 孔(hole) - 可用内存块; 各种大小的孔散布在内存中
- 当一个进程到达时, 从一个足以容纳它的孔中分配内存
- 进程退出将释放其分区, 相邻的空闲分区合并
- 操作系统维护以下信息: a) 分配的分区 b) 空闲分区 (洞)





动态分区

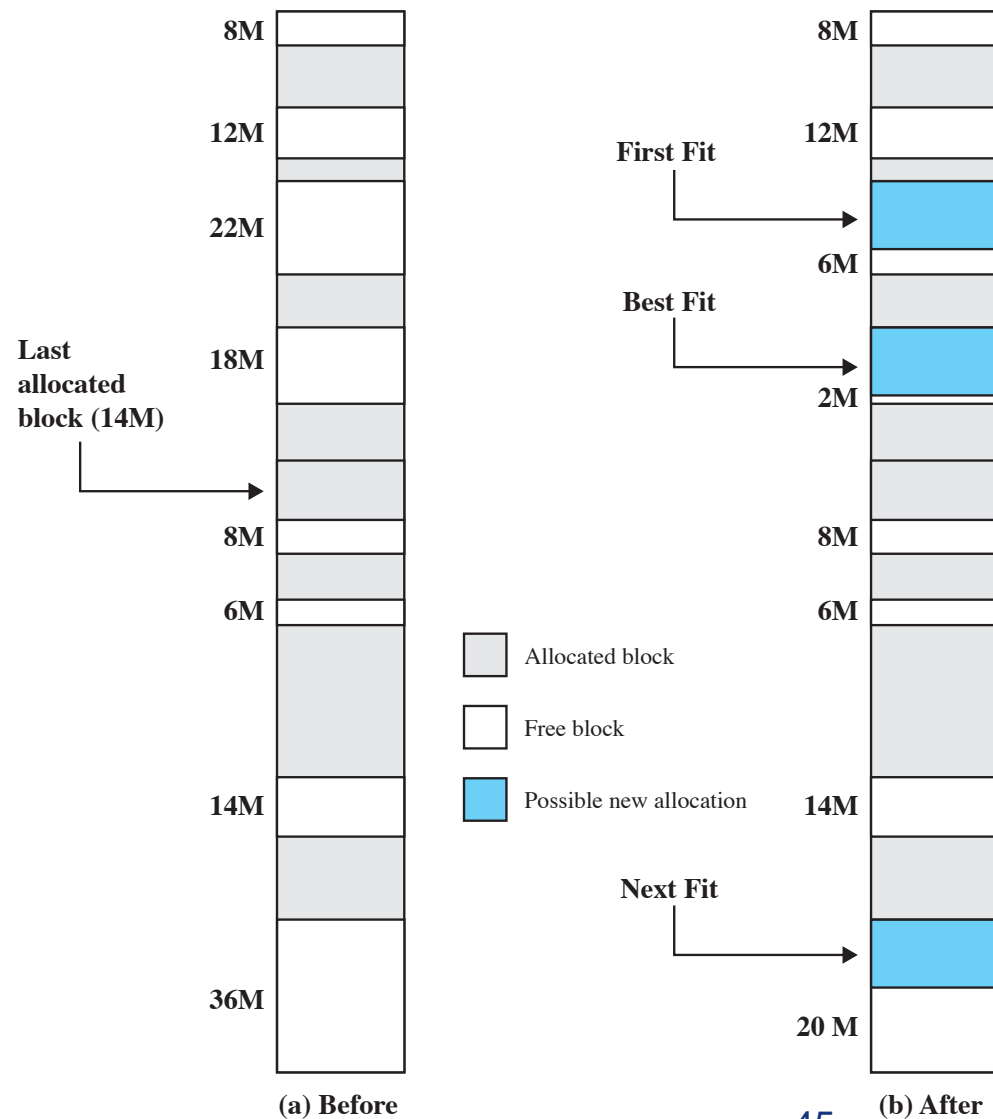
如何满足空闲孔列表中大小为 n 的请求?

- ❖ 首次适应: 分配足够大的第一个孔;
- ❖ 最佳匹配: 分配足够大的最小孔; 必须搜索整个列表, 除非按大小排序;
 - 生成最小的剩余孔
- ❖ 最差适应: 分配最大的孔; 还必须搜索整个列表;
 - 产生最大的剩余孔

在速度和存储利用率方面, First fit和best fit优于最差fit



放置算法



请求16MB

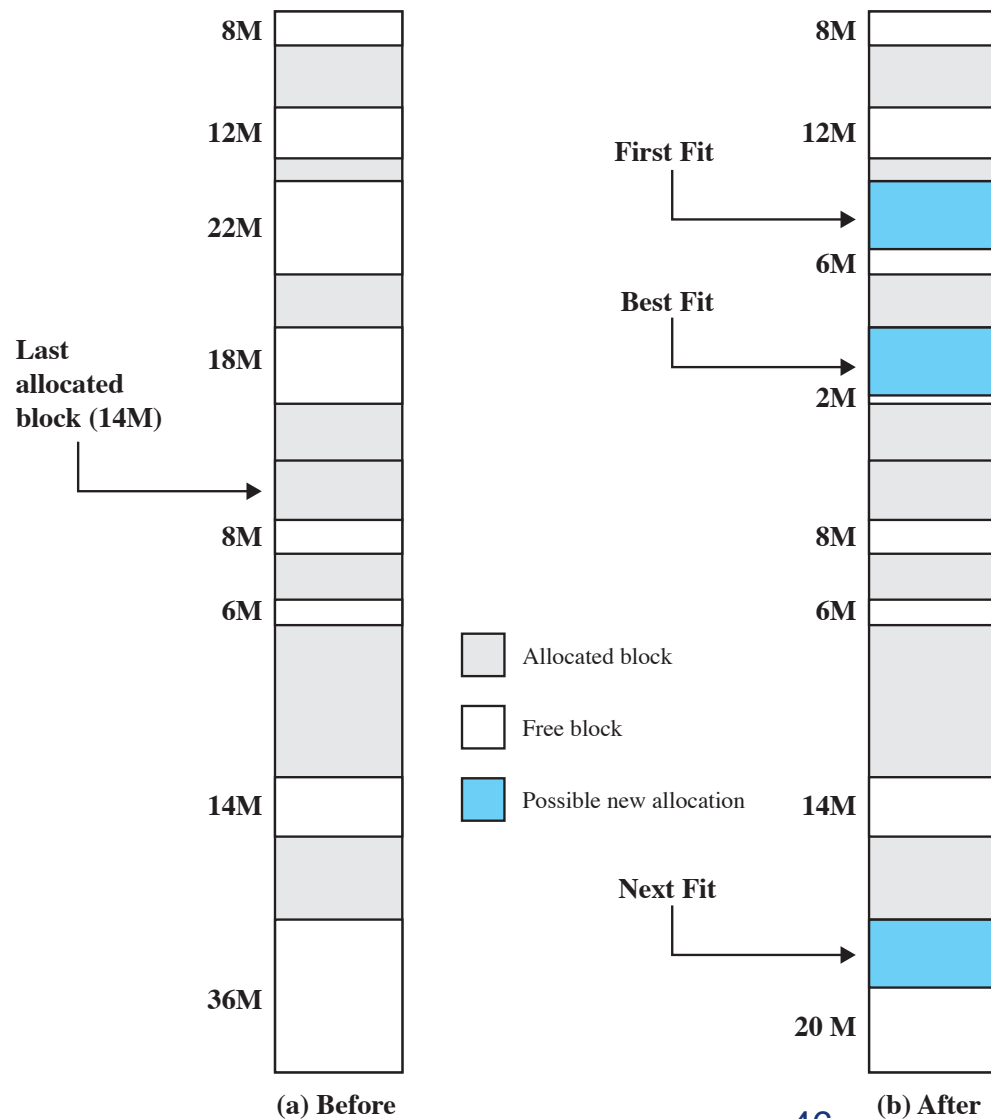
结论：首次适配算法不仅是最简单的，而且通常是最好和最快的；

首次适配算法会使得内存的前端出现很多小空闲区；

最佳适配通常性能是最差的，产生很多小到无法满足任何内存分配请求的小块，需要更频繁的内存压缩；



置换算法



- 内存中所有进程都可能阻塞；
- 操作系统将一个阻塞进程换出内存，给新进程或者处于就绪-挂起态的进程；
- 操作系统必须决定要替换哪个进程；



碎片

- ❖ 外部碎片 - 存在满足请求的总内存空间，但它不是连续的；
- ❖ 内部碎片 - 分配的内存可能略大于请求的内存；此大小差异是分区内部的内存，但未被使用；
- ❖ 选择首次适应或者最优适应可能影响碎片的数量；
- ❖ 首次适应分析表明，给定分配的 N 个块， $0.5N$ 个块因碎片而丢失
 - $1/3$ 可能无法使用 $\rightarrow 50\%$ 规则



碎片

❖ 通过压缩减少外部碎片

- 洗牌内存内容，将所有可用内存放在一个大块中
- 只有在重新定位是动态的并且在运行时完成时，才可能进行压缩；
- 外部碎片的解决方案：允许逻辑地址不连续，分段和分页；

❖ 考虑磁盘有相同的碎片问题；



分段

- ❖ 程序和其相关的数据划分到几个段 (segment)
 - 所有段的长度不一定相等;
 - 段有最大长度;
- ❖ 逻辑地址也有两部分构成:
 - 段号
 - 偏移量
- ❖ 分段似于动态分区;
- ❖ 需要将程序的所有分段都装入内存;
- ❖ 一个程序占据多个分区, 并且这些分区不要求是连续的;
- ❖ 消除了内部碎片, 但会产生外部碎片;





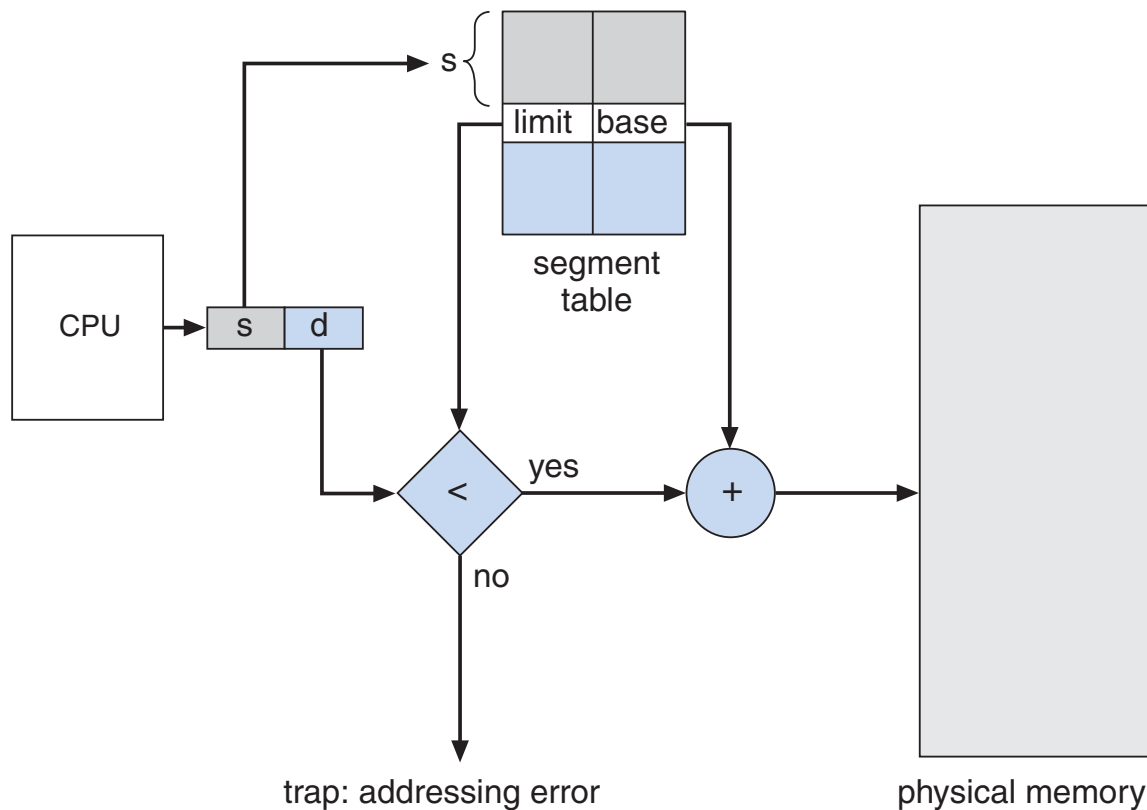
分段

- ❖ **Usually visible** (可见的) ;
- ❖ 作为组织程序和数据的一种方便手段提供给程序员;
- ❖ 程序员或者编译器会把程序和数据指定到不同的分段;
- ❖ 为了实现模块化程序设计的目的, 程序和数据可能会进一步划分成多个段; 每个段都有名称和长度:
 - 这种方法最大的不方便是程序需要知道段的最大长度限制;



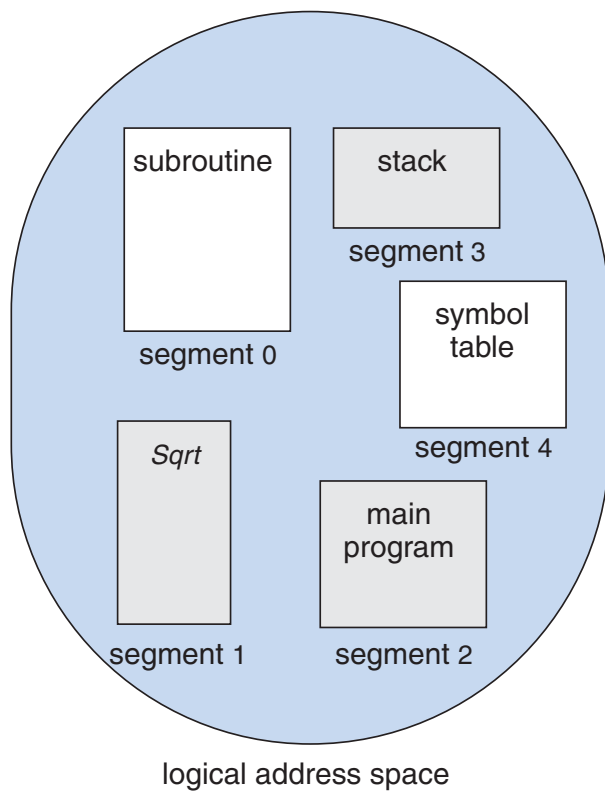
分段硬件

- ❖ 将程序产生的二维地址即<段号、偏移>映射到一维物理地址，这个地址是通过段表实现；
- ❖ 段表中每个条目由段基址和段界限构成；



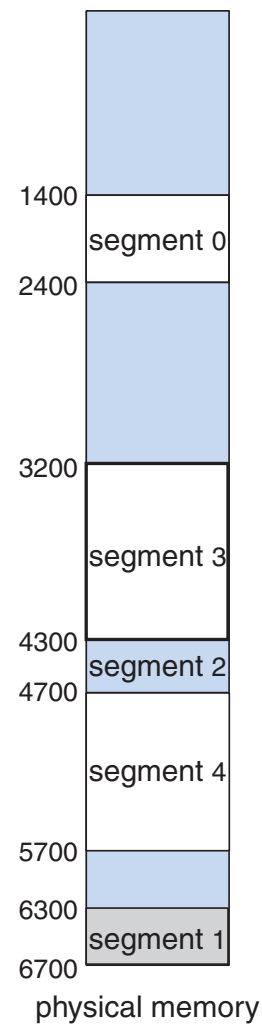


分段举例



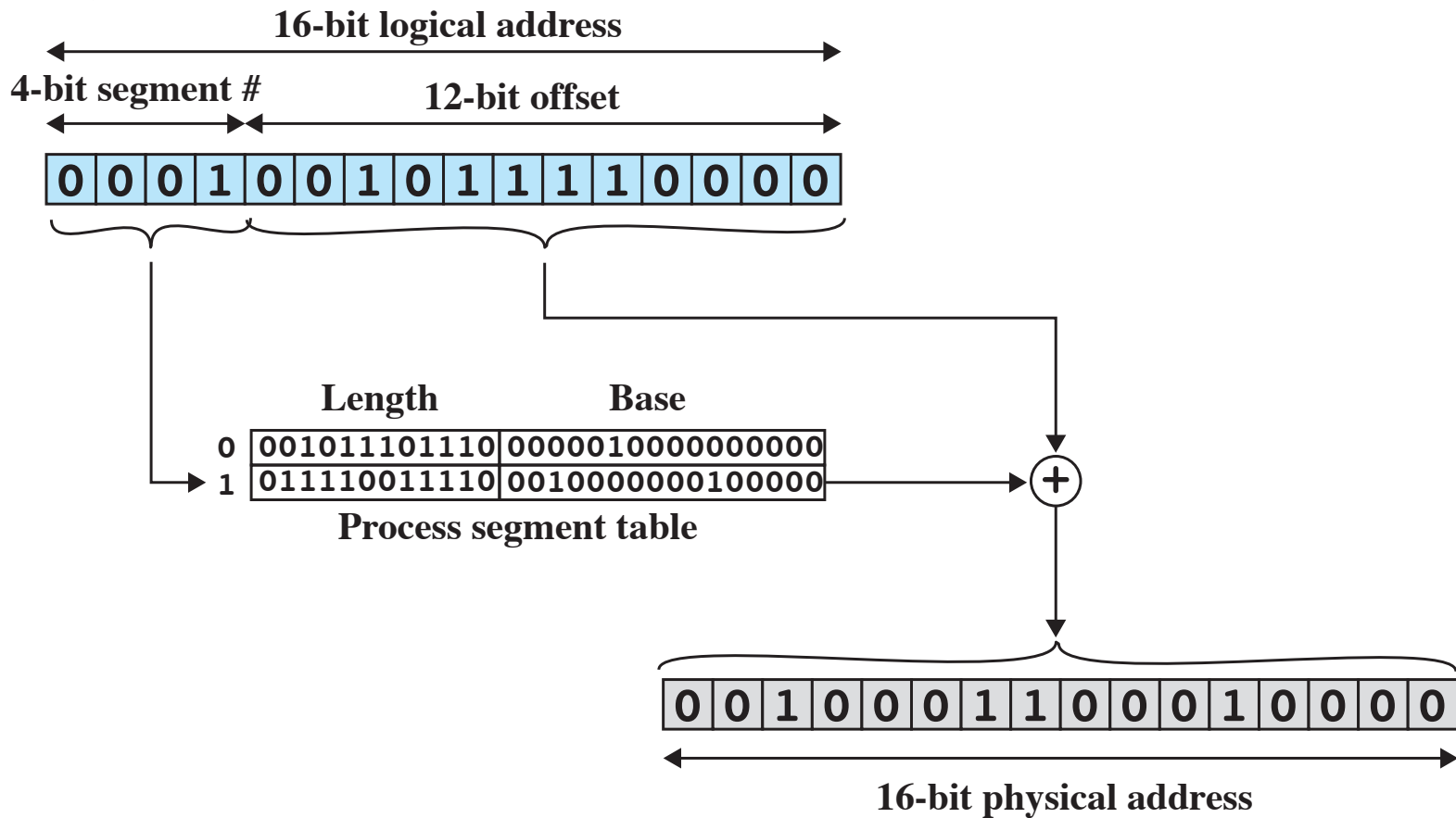
	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table





地址转换



(b) Segmentation



分页

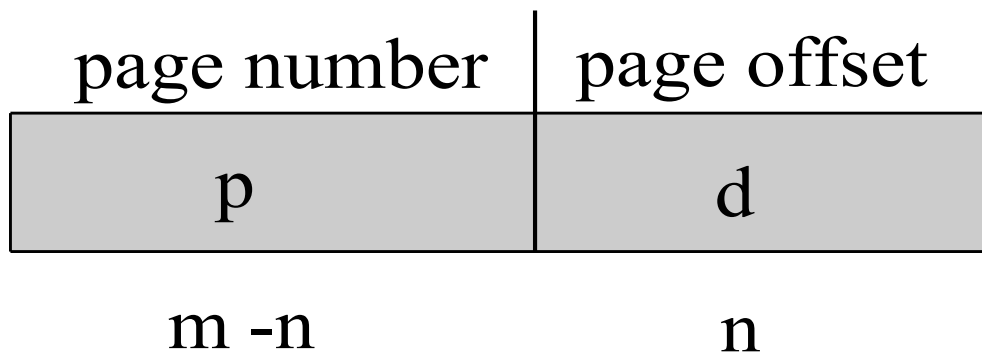
- ❖ 进程的物理地址空间可以是不连续的；只要物理内存可用，就为进程分配物理内存
 - 避免外部碎片
 - 避免了大小不一的内存块问题
- ❖ 将物理内存划分为称为帧的固定大小的块
 - 大小是2的幂，介于512字节和16 MB之间
- ❖ 将逻辑内存划分为大小相同的块（称为页）
- ❖ 跟踪所有空闲帧；
- ❖ 要运行大小为N页的程序，需要找到N个空闲帧并加载程序
- ❖ 设置页表以将逻辑地址转换为物理地址；
- ❖ 备份存储区（磁盘）同样拆分为多个页面；
- ❖ 仍然有内部碎片；



地址转换

❖ CPU生成的地址分为:

- 页码 (p) - 用作页表的索引, 该页表包含物理内存中每页的基址
- 页偏移量 (d) - 与基址相结合, 定义发送到内存单元的物理内存地址

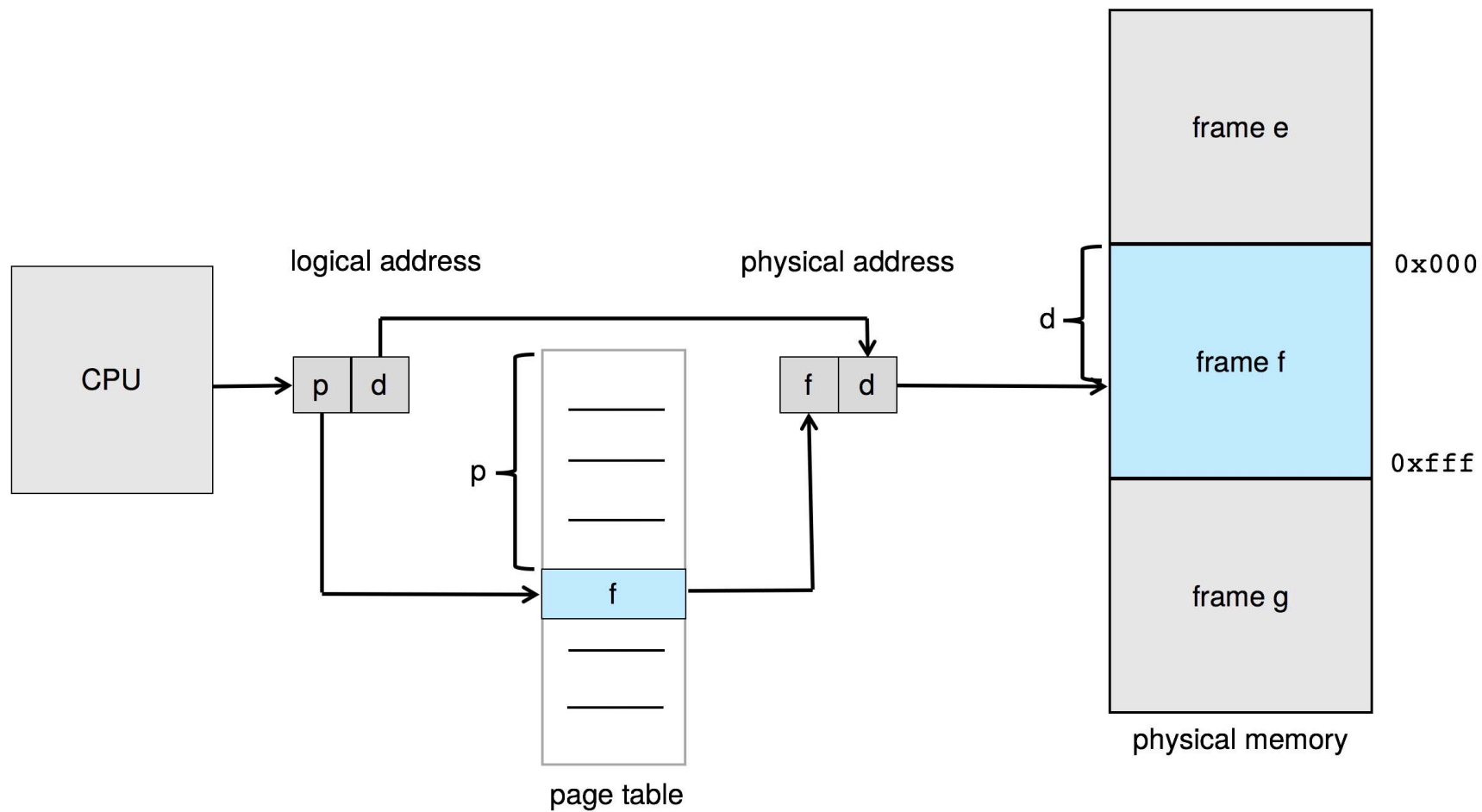


- 对于给定的逻辑地址空间 2^m 和页面大小 2^n

Getconf PAGESIZE

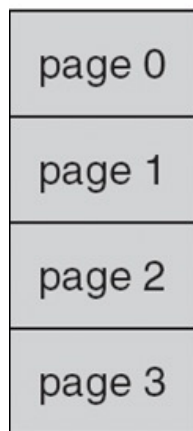


分页硬件





逻辑和物理内存分页模型

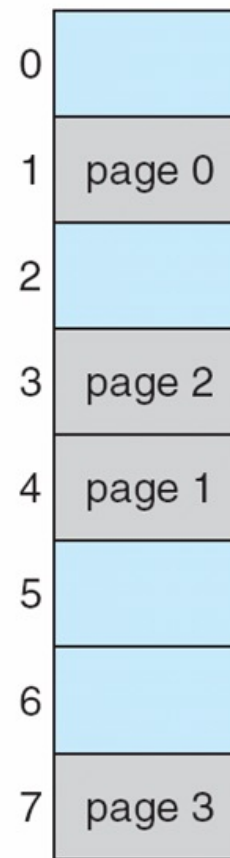


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number



physical
memory



分页示例

- ❖ 逻辑地址: $n=2$, $m=4$ 。使用4字节的页面大小和32字节 (8页) 的物理内存

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

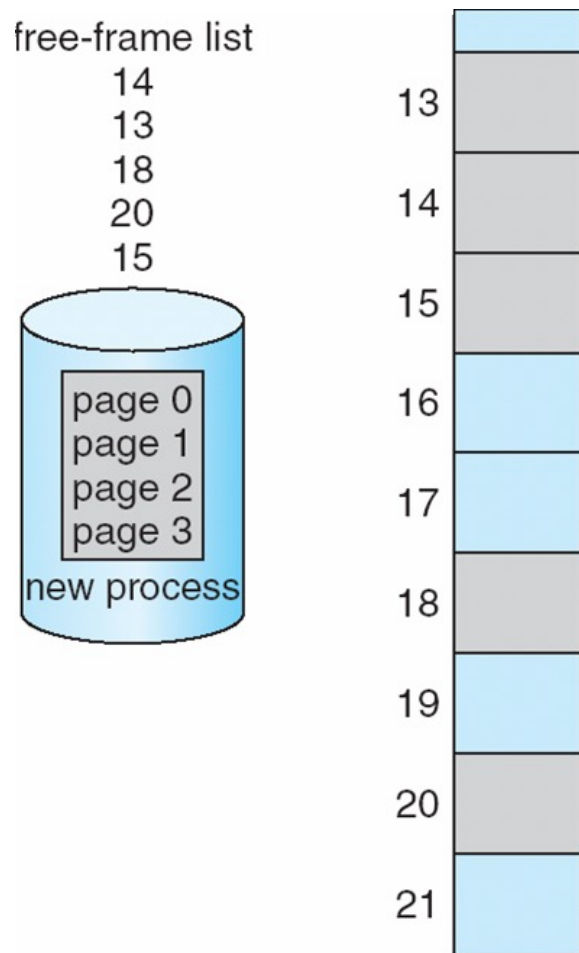


计算内部碎片

- ❖ 页面大小=2048字节
- ❖ 进程大小=72766字节
- ❖ 35页+1086字节
- ❖ $2048 - 1086 = 962$ 字节的内部碎片
- ❖ 最坏情况碎片=1帧-1字节
- ❖ 平均碎片=1/2帧大小
- ❖ 这么小的框架尺寸合适吗?
- ❖ 但是每个页表条目都需要内存来跟踪
- ❖ 页面大小随时间增长
 - Solaris支持两种页面大小-8KB和4MB

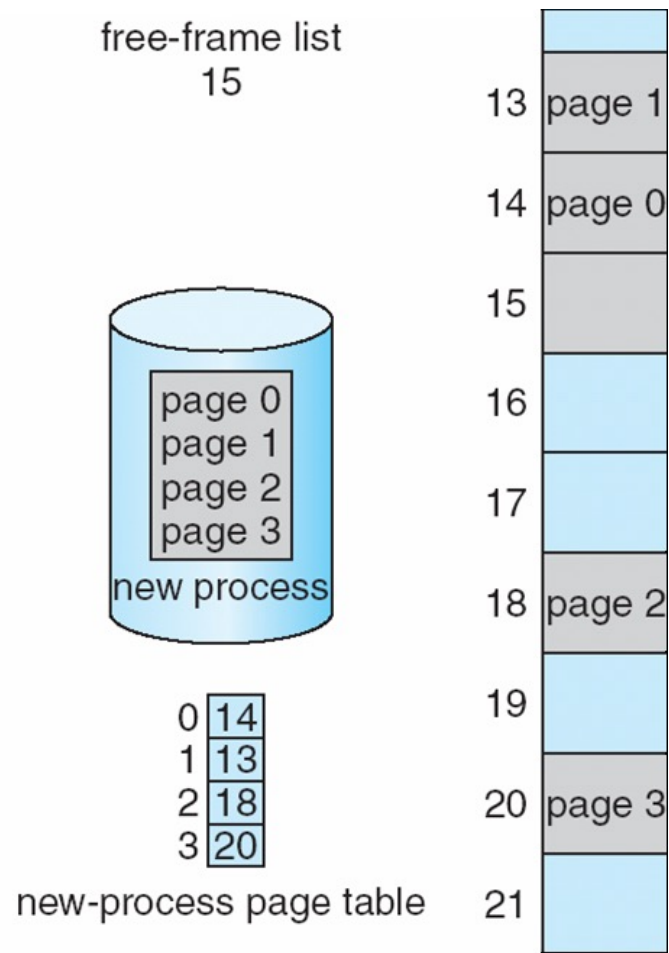


空闲帧



(a)

分配前



(b)

分配后



页表的实现

- ❖ 页表保存在主存中，为每一个进程分配；
 - 页表基址寄存器 (PTBR) 指向页表
 - 页表长度寄存器 (PTLR) 指示页表的大小
 - 特权指令；
- ❖ 在该方案中，每个数据/指令访问需要两次内存访问
 - 一个用于页表，一个用于数据/指令
- ❖ 两次内存访问问题可以通过使用一种特殊的快速查找硬件缓存（称为转换查找缓冲区 (TLB)（也称为关联内存））来解决。
- ❖ TLB是键-值存储，给定值查找时，同时与所有的键进行比较；
- ❖ 多层的TLB设计，用于性能加速；



TLB (转换表缓冲区)

- ❖ 一些TLB在每个TLB条目中存储地址空间标识符 (ASID) ——唯一地标识每个进程，为该进程提供地址空间保护
 - 否则需要在每个上下文切换时刷新；
- ❖ TLB通常较小 (64到1024个条目)
- ❖ 在TLB未命中时，值将加载到TLB中，以便下次更快地访问
 - 必须考虑更换策略；
 - 一些条目可以固定下来，以便永久快速访问



TLB (转换表缓冲区, 快表)

❖ 联想记忆-并行搜索

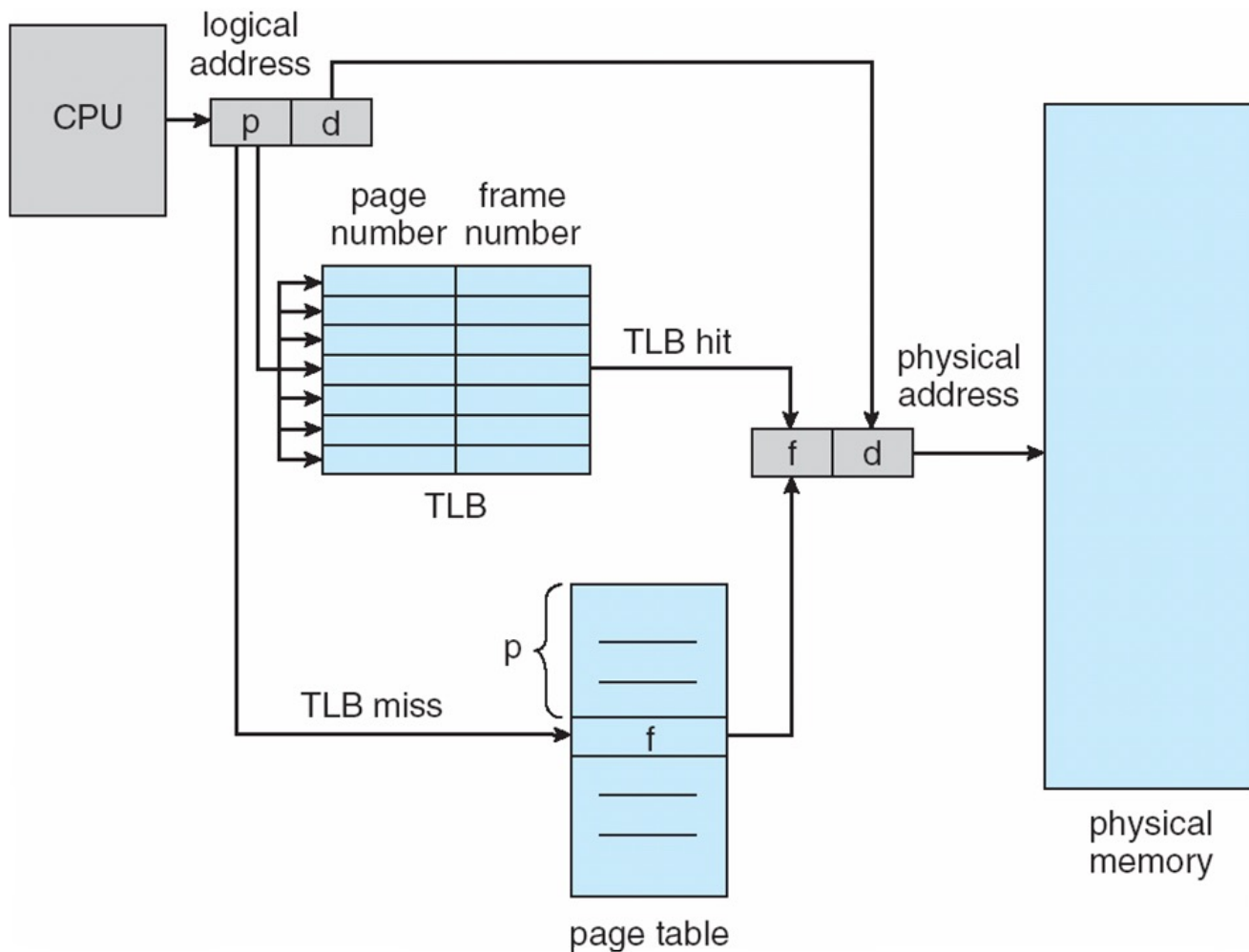
Page #	Frame #

❖ 地址转换 (p, d)

- 如果p在关联寄存器中, 则将帧号取出
- 否则, 从内存中的页表中获取帧号



带TLB的分页硬件





有效访问时间

- 命中率 - 在TLB中找到页码的百分比
- ❖ 80%的命中率意味着我们80%的时间在TLB中找到所需的页码。
- ❖ 假设访问内存需要10纳秒。
 - 如果我们在TLB中找到所需的页面，则映射内存访问需要10 ns
 - 否则，我们需要两个内存访问，所以它是20纳秒

❖ 有效访问时间 (EAT)

$$EAT = 0.80 \times 10 + 0.20 \times 20 = 12 \text{ 纳秒}$$

意味着访问时间减少20%

❖ 考虑到现实的命中率为99%，

$$EAT = 0.99 \times 10 + 0.01 \times 20 = 10.1 \text{ ns}$$

这意味着访问时间仅减少1%。

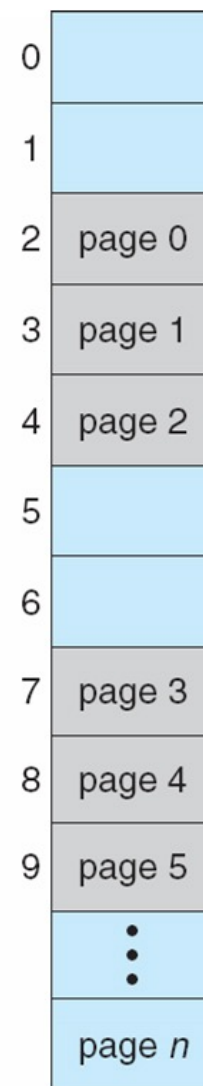
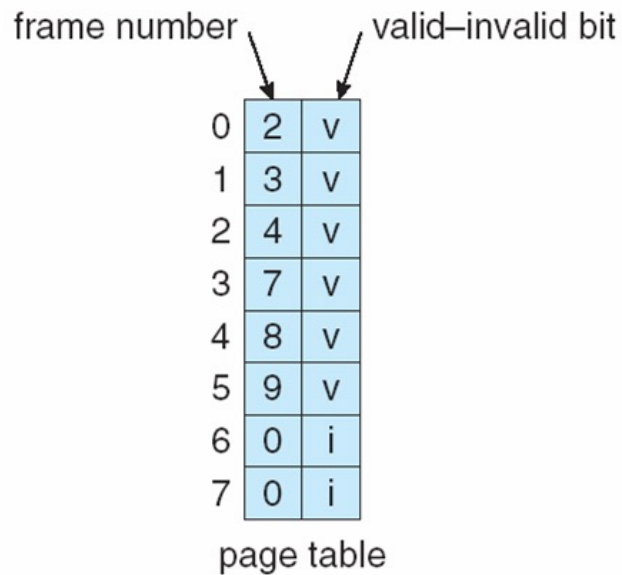
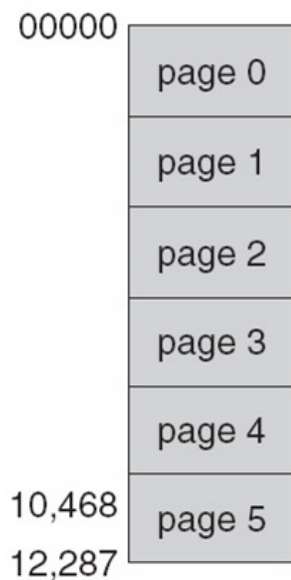


内存保护

- ❖ 通过将保护位与每个帧关联以指示是否允许只读或读写访问来实现内存保护
 - 还可以添加更多位以指示仅执行页面，依此类推
- ❖ 附加到页表中每个条目的有效-无效位：
 - “valid” 表示关联页位于进程的逻辑地址空间中，因此是合法页；
 - “invalid” 表示页不在进程的逻辑地址空间中
 - 或使用页表长度寄存器（PTLR），表示页表大小，用于检查每个逻辑地址是否有效；
- ❖ 任何违规行为都会导致内核陷阱



页表中的有效 (v) 位或无效 (i) 位





共享页

❖ 共享代码

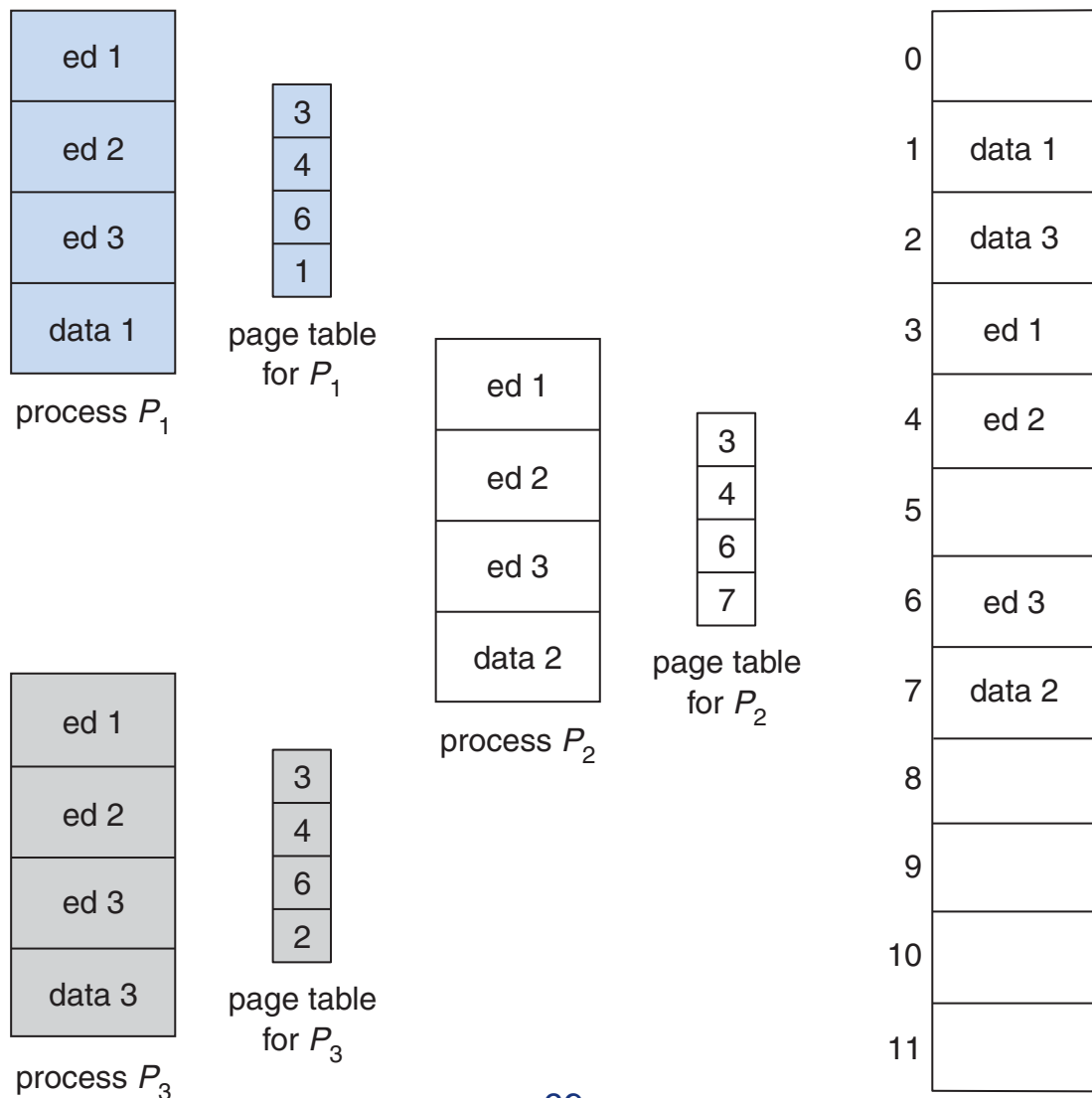
- 进程（即文本编辑器、编译器、窗口系统）之间共享的只读（可重入）代码的一个副本；
- 类似于共享同一进程空间的多个线程；
- 如果允许共享读写页面，则还可用于进程间通信；

❖ 私有代码和数据

- 每个进程都保留代码和数据的单独副本
- 私有代码和数据的页面可以出现在逻辑地址空间的任何位置



共享页举例





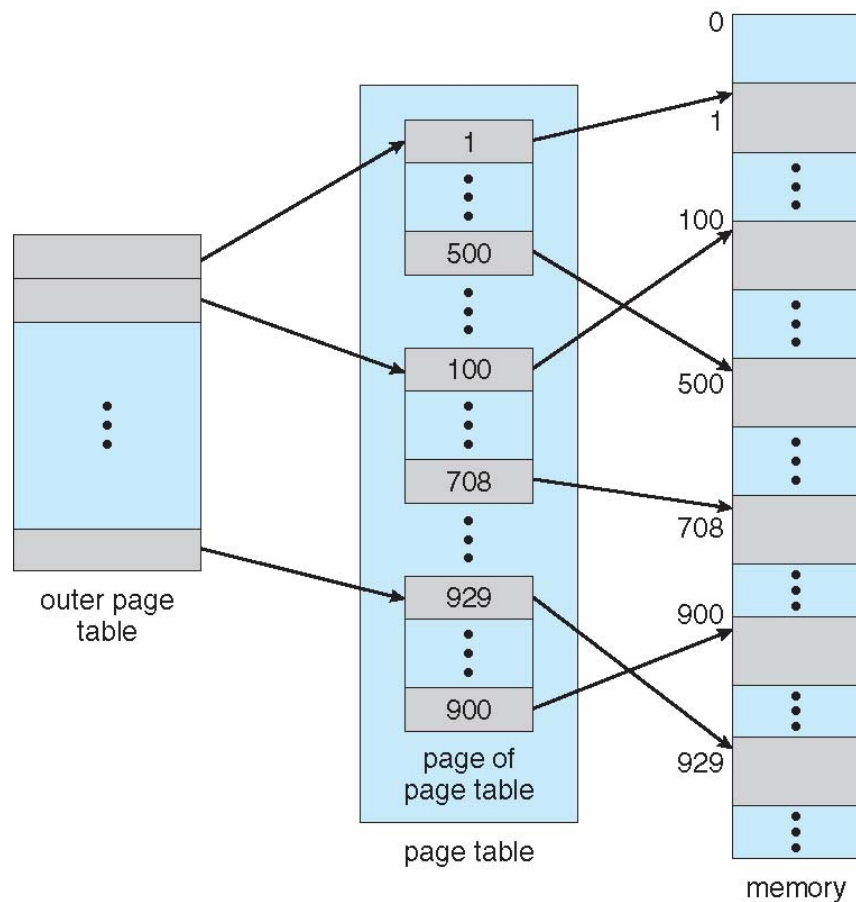
页表结构

- ❖ 使用直接的方法，分页的内存结构可能会变得非常庞大
 - 在现代计算机上考虑32位逻辑地址空间
 - 页面大小为4KB (2^{12})
 - 页面表将有100万个条目 ($2^{32}/2^{12}$)
 - 如果每个条目为4字节，每个进程为页表分配4 MB的物理地址空间
 - 不希望在主内存中连续分配该内存；
 - 一个简单的解决方案是将页表划分为更小的单元
 - 分层分页；
 - 散列页表；
 - 倒排页表；



分层页表

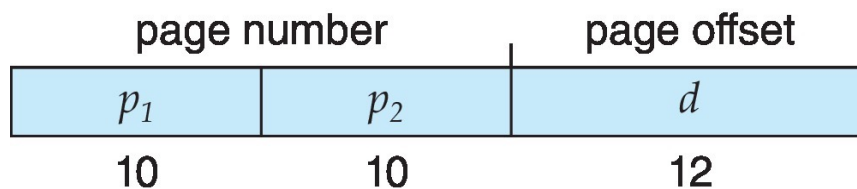
- ❖ 将逻辑地址空间拆分为多个页表;
- ❖ 一种简单的技术是两级页表;
- ❖ 然后我们分页页表;





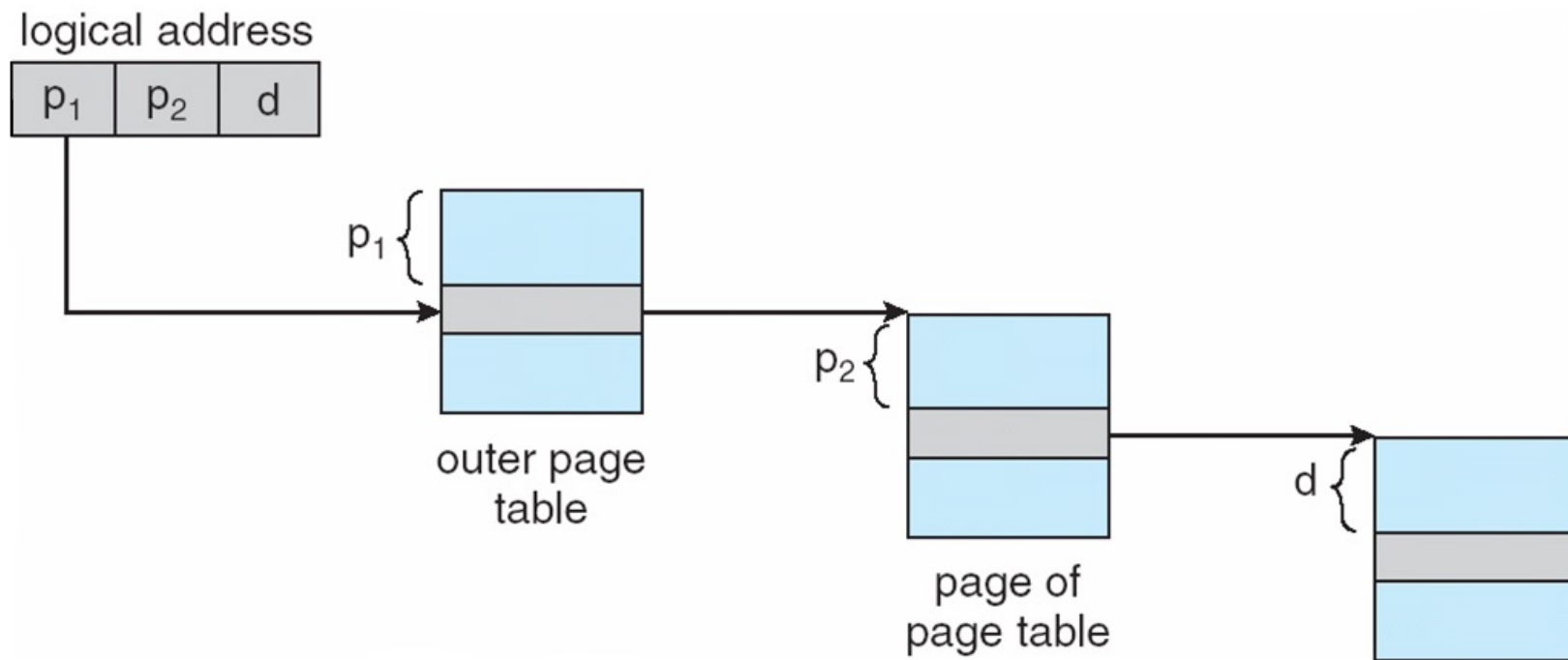
分层页表

- ❖ 逻辑地址（在具有4K页面大小的32位机器上）分为：
 - 由20位组成的页码
 - 由12位组成的页偏移量
- ❖ 由于页表是分页的，因此页码进一步分为：
 - 10位页码
 - 10位页偏移量
- ❖ 因此，逻辑地址如下所示：
- ❖ 其中 p_1 是外部页面表的索引， p_2 是内部页面表页面内的位移
- ❖ 称为前向映射页表





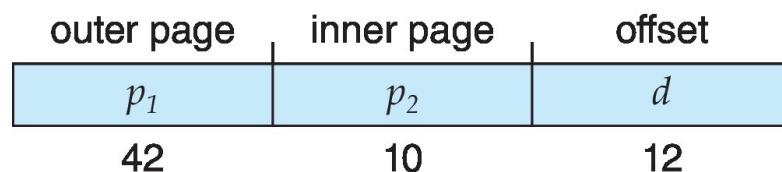
地址转换方案





64位逻辑地址空间

- ❖ 即使是两级分页方案也不够
- ❖ 如果页面大小为4KB (2^{12})
 - 然后页面表有 2^{52} 个条目
 - 如果是两级方案，则内部页表可以是 2^{10} 个4字节的条目
 - 地址看起来像



- 外部页表有 2^{42} 个条目或 2^{44} 个字节
- 一种解决方案是添加第二个外部页面表
- 但在下面的示例中，第二个外部页表的大小仍然是 2^{34} 字节
 - 可能需要4次内存访问才能访问一个物理内存位置



三级分页

outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

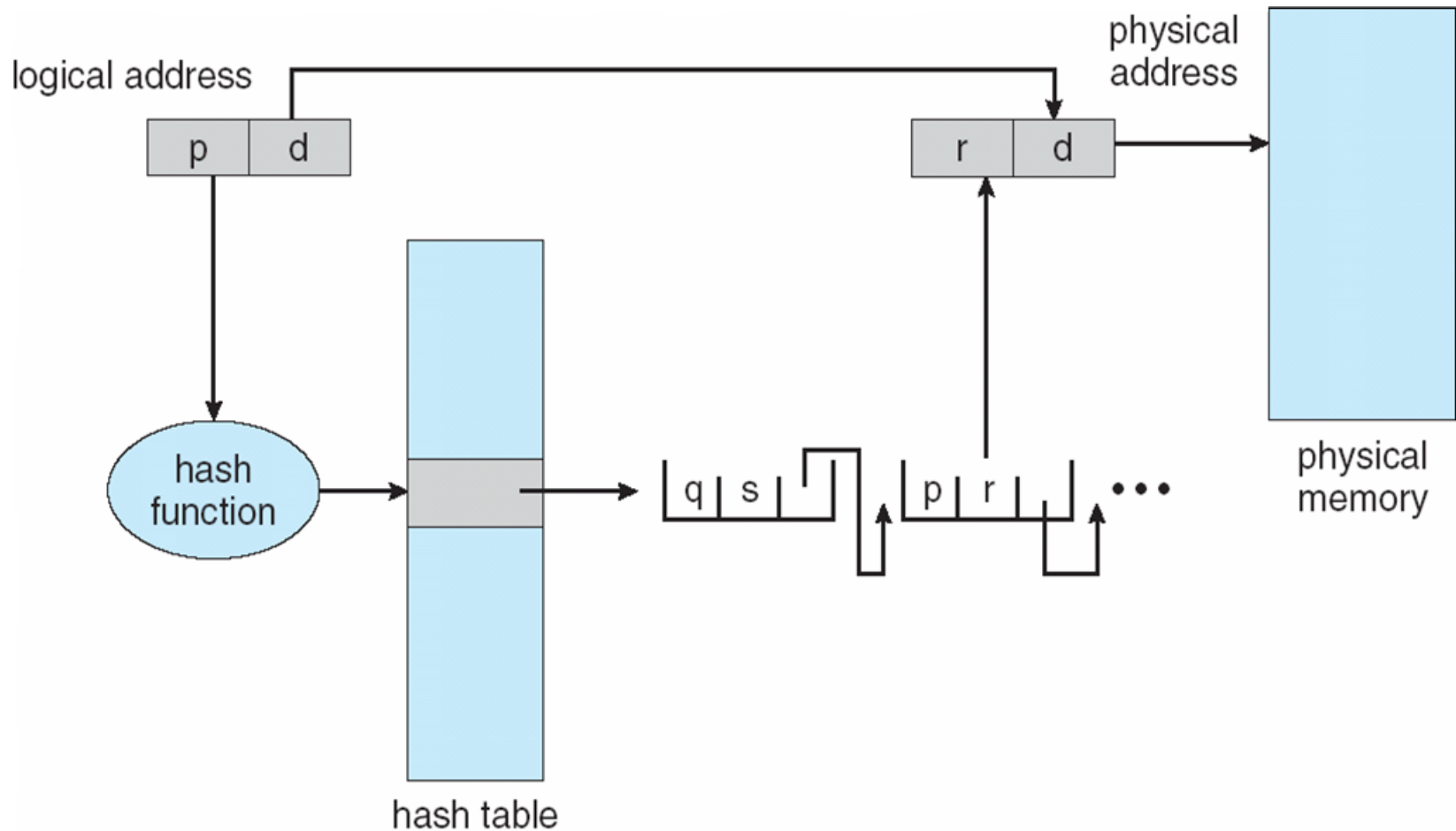


散列页表 (哈希页表)

- ❖ 公共地址空间 >32 位;
- ❖ 虚拟页码散列到页表中
 - 虚拟页码作为哈希值, 此页表包含散列到同一位置的元素链
- ❖ 每个元素都包含 (1) 虚拟页码 (2) 映射的页帧码 (3) 指向下一个元素的指针
- ❖ 在此链中比较虚拟页码以搜索匹配项
 - 如果找到匹配, 则提取相应的物理帧, 如果不匹配, 查找链表的下一个元素;
- ❖ 64位地址的变体是聚簇页表
 - 与哈希相似, 但每个条目引用多个页面 (如16页), 而不是1页
 - 特别适用于稀疏地址空间 (内存引用不连续且分散)



散列页表



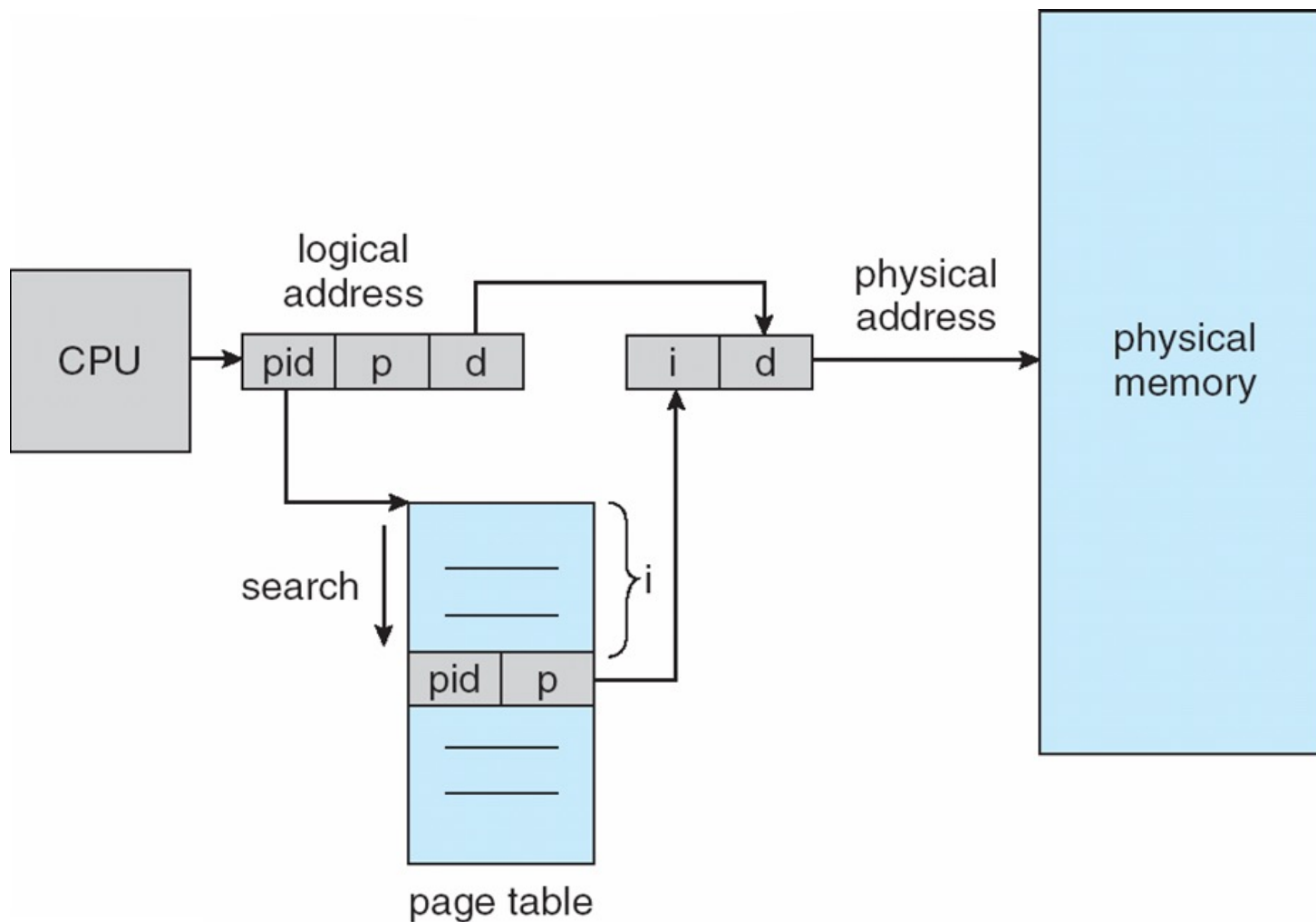


倒排页表

- ❖ 与其每个进程都有一个页表并跟踪所有可能的逻辑页，不如跟踪所有物理页
- ❖ 内存的每一物理页有一个条目；
- ❖ 条目由存储在该实际内存位置的页的虚拟地址以及有关拥有该页的进程的信息组成
- ❖ 减少存储每个页表所需的内存，但在发生页引用时增加搜索表所需的时间
- ❖ 使用哈希表将搜索限制为一页或最多几页的表项
 - TLB可以加速访问
- ❖ 但是如何实现共享内存呢？
 - 虚拟地址到共享物理地址的一种映射



倒排页表





Oracle SPARC Solaris

- ❖ 考虑现代64位操作系统实例
 - 目标是效率，低开销
- ❖ 基于哈希，但更复杂；
- ❖ 两个哈希表
 - 一个内核，一个用于所有用户进程
 - 每个都将内存地址从虚拟内存映射到物理内存
 - 每个条目表示映射虚拟内存的连续区域，
 - 比为每个页面提供单独的哈希表条目更高效
 - 每个条目都有基址和跨距（表示条目所代表的页数）



Oracle SPARC Solaris

- ❖ TLB保存用于快速硬件查找的转换表条目 (TTE)
 - TTE缓存驻留在转换存储缓冲区 (TSB) 中
 - 每个最近访问的页面包含一个条目
- ❖ 虚拟地址引用导致TLB搜索
 - 如果未命中，硬件将遍历内存中的TSB，查找与地址对应的TTE
 - 如果找到匹配项，CPU将TSB条目复制到TLB中并完成转换
 - 如果没有找到匹配项，内核将中断以搜索哈希表
 - 然后内核从适当的哈希表创建一个TTE并将其存储在TSB中，中断处理程序将控制权返回给MMU，MMU完成地址转换。



示例：英特尔32位和64位体系结构

- ❖ 主流的产业芯片
- ❖ 奔腾CPU是32位的，称为IA-32体系结构
- ❖ 当前的英特尔CPU是64位的，称为IA-64体系结构
- ❖ 芯片中的许多变体涵盖了这里的主要思想



示例：英特尔32位和64位体系结构

❖ 支持分段和分页分段

- 每个段可以是4 GB
- 每个进程最多16 K段
- 分成两部分
 - 最多8K段的第一个分区是私有的（保存在本地描述符表（LDT）中）
 - 在所有进程之间共享的最多8K段的第二个分区（保存在全局描述符表（GDT）中）



示例：英特尔32位和64位体系结构

❖ CPU生成逻辑地址

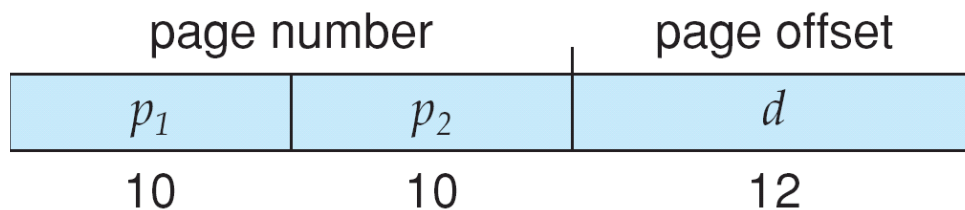
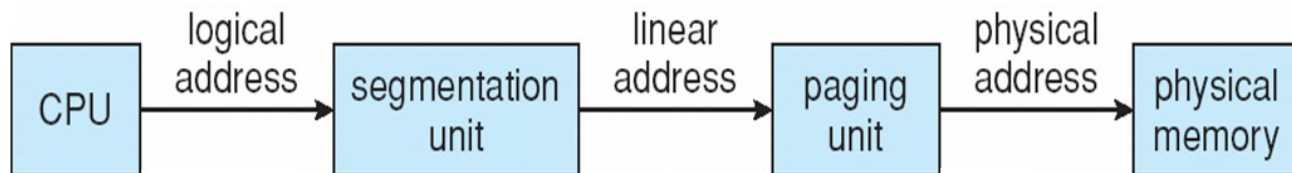
- 给分段单元的选择器
 - 它产生线性地址

<i>s</i>	<i>g</i>	<i>p</i>
13	1	2

- 分页单元的线性地址
 - 它的主存中生成物理地址
 - 分页单元相当于MMU
 - 页面大小可以是4KB或4MB

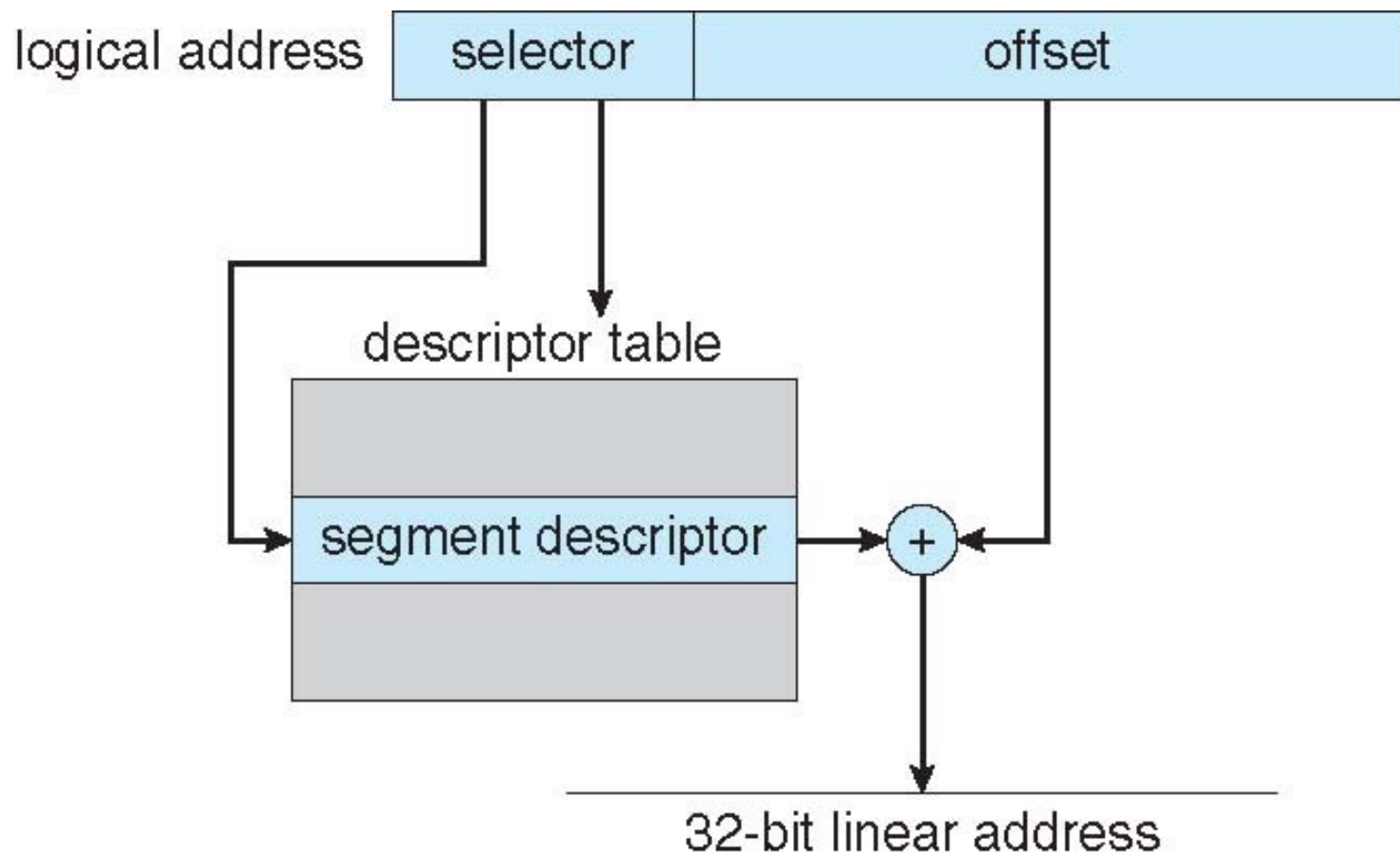


IA-32中的逻辑到物理地址转换



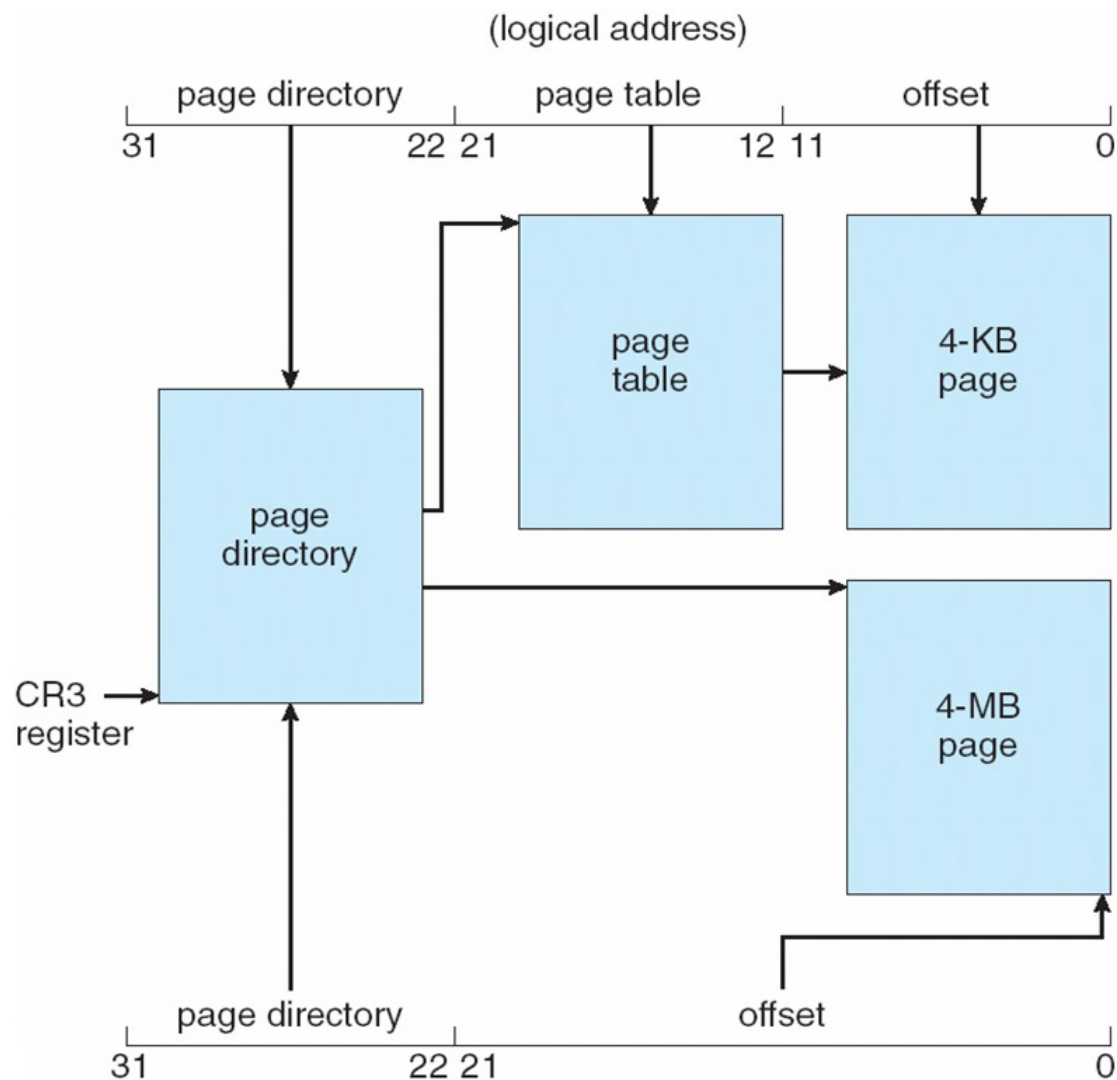


英特尔IA-32分段





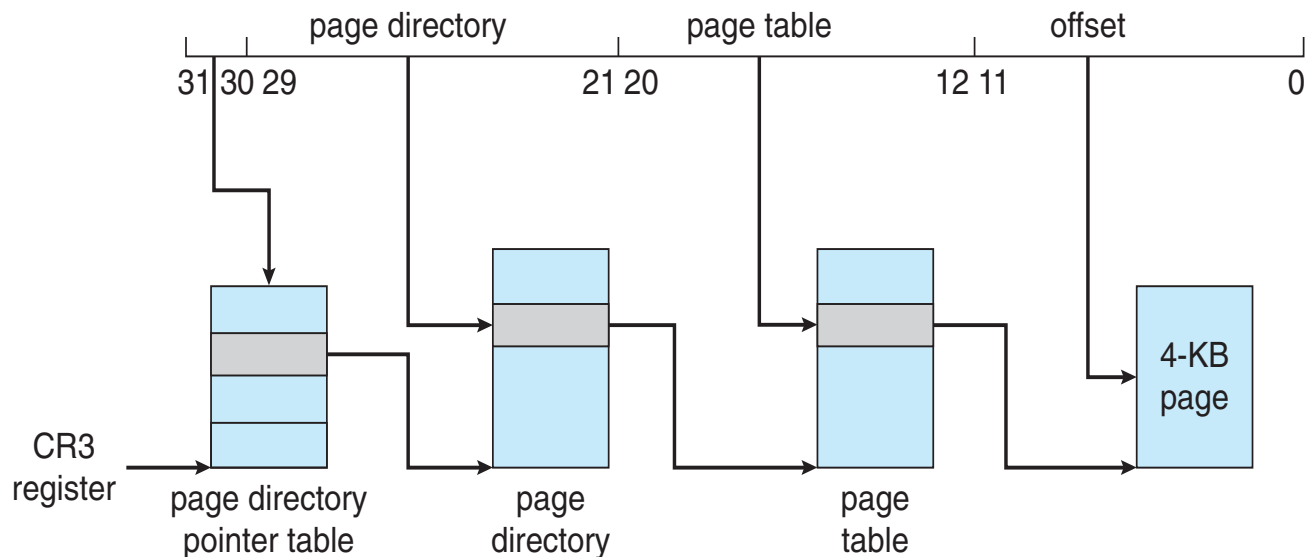
英特尔IA-32分页体系结构





英特尔IA-32页地址扩展

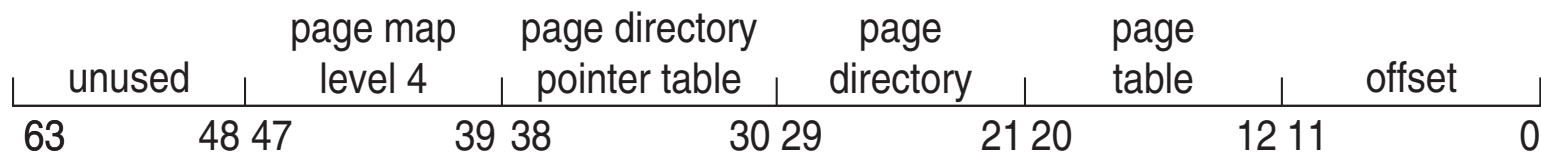
- 32位地址限制促使Intel创建页面地址扩展 (PAE)，允许32位应用程序访问超过4GB的内存空间
 - 分页转到了3级方案
 - 前两位表示页目录指针表
 - 页目录和页表条目的大小移动到64位
 - 效果是将地址空间增加到36位 - 64GB的物理内存





英特尔IA-32页地址扩展

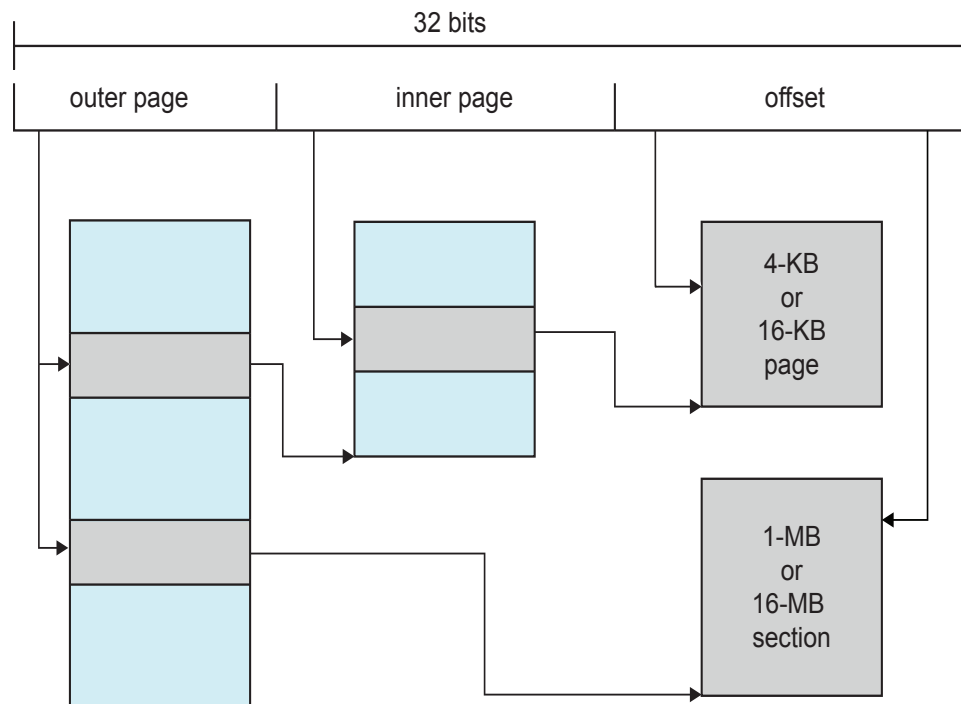
- 当前一代英特尔x86体系结构
- 64位为ginormous (>16 EB)
- 实际上, 仅实现48位寻址
 - 页面大小为4KB、2MB、1GB
 - 分页层次结构的四个级别
- 还可以使用PAE, 使虚拟地址为48位, 物理地址为52位





示例：ARM体系结构

- 主流的移动平台芯片（例如苹果iOS和谷歌安卓设备）
- 现代、节能、32位CPU
- 4KB和16KB页面
- 1 MB和16 MB页面（称为段）
- 一级分页用于分段，两级分页用于较小的页面
- 两级TLB
 - 外层有两个微型TLB（一个数据，一个指令）
 - 内部为单主TLB
 - 首先检查微TLB，然后检查主TLB，仍未找到则通过硬件查找页表；





伙伴系统 (Buddy System)

- ❖ 动态分区和固定分区都存在明显的缺点，伙伴系统融合了固定和动态分区；
- ❖ 可用于分配的整个空间看成一个块；
- ❖ Memory blocks are available of size 2^K words, $L \leq K \leq U$, where
 - 2^L = smallest size block that is allocated
 - 2^U = largest size block that is allocated; generally 2^U is the size of the entire memory available for allocation



伙伴系统 (Buddy System)

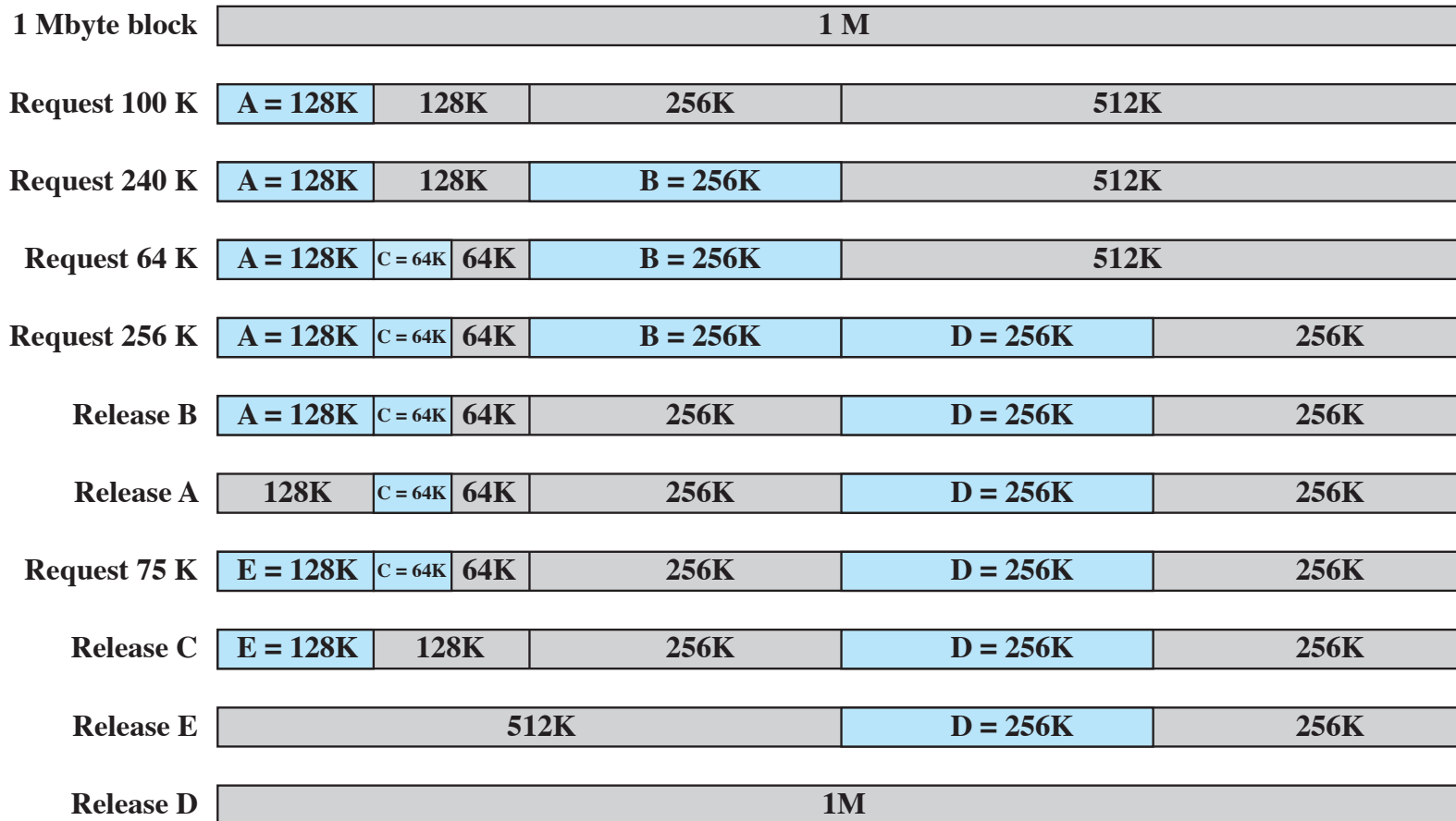


Figure 7.6 Example of Buddy System



伙伴系统的树状表示

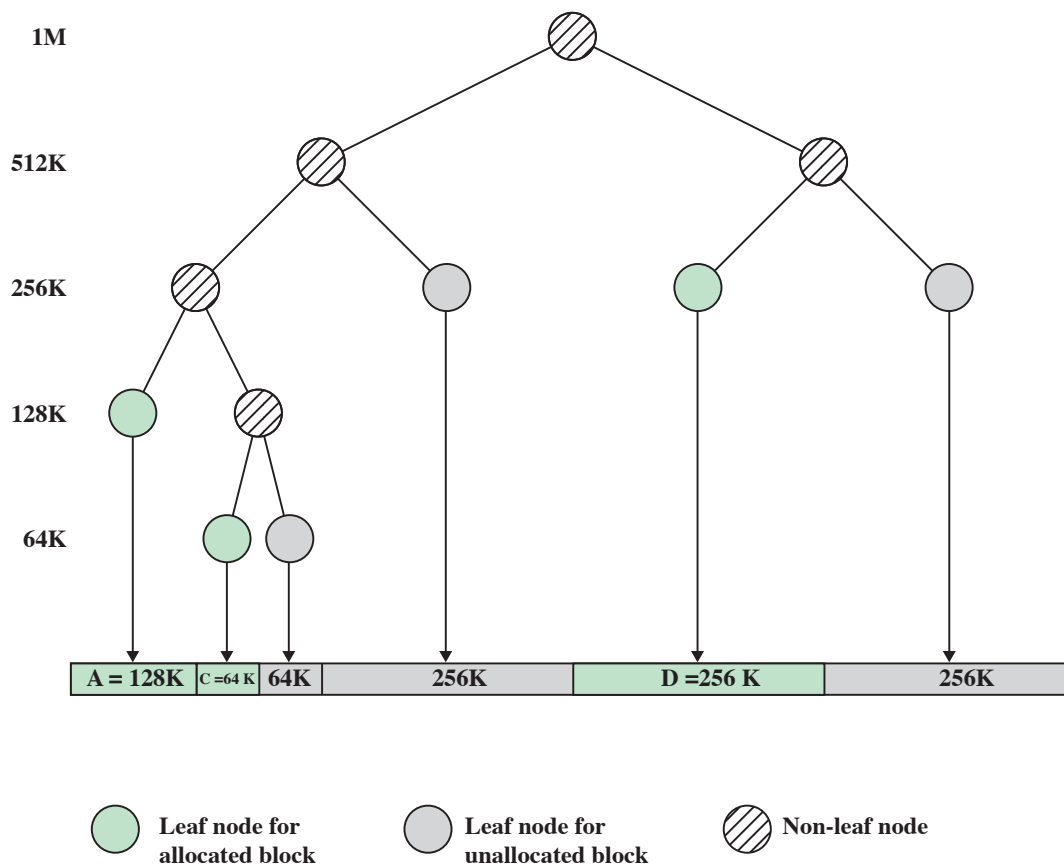


Figure 7.7 Tree Representation of Buddy System



中山大學
SUN YAT-SEN UNIVERSITY

计算机学院 (软件学院)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



谢谢